

- 1. AI Usage in Documents (PA2)
- 2. AI Usage in Implementing Code to Complete Features:
  - Kim Hằng's AI Usage:
  - Gia Phúc's AI Usage:
  - Khánh Linh's AI Usage:
  - Thiên Phước's AI Usage:
  - Minh Thư's AI Usage:
- Appendix: AI Usage Proof
  - 1
    - 1.1
  - 2
    - 2.1
    - 2.2
    - 2.3
    - 2.4
    - 2.5
    - 2.6
    - 2.7
    - 2.8
    - 2.9
    - 2.10
    - 2.11
    - 2.12
    - 2.13
    - 2.14
    - 2.15
    - 2.16

## 1. AI Usage in Documents (PA2)

---

- *Gemini*. Gemini 1.5, Google, gemini.google.com, accessed 11:43 on July 9, 2026, prompt: "What is the Stakeholder and Users description? How to know what type of stakeholder and user in a project?", used to refine and structure the project documentation based on hand-written notes and bullet points. The team provided rough ideas for sections such as the project overview, requirements, risks, and sprint plan, and Gemini helped transform them into smoother, more formal paragraphs with clearer grammar and better flow. In several cases, it was also used to suggest additional supporting details and improve the logical organization of the report. All final content was reviewed and adjusted by the team to ensure accuracy and alignment with the actual project scope.

**Chat History:** See in Appendix 1.1 [proof](#)

## 2. AI Usage in Implementing Code to Complete Features:

---

Kim Hằng's AI Usage:

- *ChatGPT*. GPT-4o, OpenAI, chatgpt.com, accessed 10:43 on June 13, 2026, prompt: "What is the difference between writing API documentation before vs after coding (API-first vs Code-first approach) and what does the dataValues field mean in a Sequelize model instance?", used to understand API architecture design patterns and Sequelize internals; AI generated explanations of both concepts, student validated by comparing with official Sequelize documentation and applied the API-first approach to planning the backend routes.

**Chat History:** See in Appendix 2.1

- *ChatGPT*. GPT-4o, OpenAI, chatgpt.com, accessed 10:43 on June 17, 2026, prompt: "How can I test an API (e.g. difference between pasting requests in terminal vs tools like Thunder Client/Swagger) and how do JWT tokens work in an authentication flow?", used to design the API testing strategy and security layers; AI explained visual API clients and the access/refresh token mechanism, student set up Thunder Client in VS Code and implemented the Authorization headers in local requests.

**Chat History:** See in Appendix 2.2

- *ChatGPT*. GPT-4o, OpenAI, chatgpt.com, accessed 10:43 on June 20, 2026, prompt: "Why are there two register routes (/user/register and /auth/register), how do I separate responsibilities between auth and user modules, and why does my register response return 'Login successfully'?", used to clean up redundant routes and fix the response JSON message; AI explained controller separations and internal auth.service calls, student deleted the redundant /user/register route and customized the response status to return "Register successfully".

**Chat History:** See in Appendix 2.3

- *ChatGPT*. GPT-4o, OpenAI, chatgpt.com, accessed 10:43 on June 27, 2026, prompt: "What is the difference between JWT Guard and Email Verification, is JWT Guard necessary, and what are the two approaches to reset password (6-digit OTP vs reset token)?", used to secure endpoints and choose a reset password architecture; AI compared authentication vs email validity and detailed OTP vs Reset Token logic, student implemented the JWT Guard on all protected routes and chose the 6-digit OTP approach for simple form verification.

**Chat History:** See in Appendix 2.4

- *ChatGPT*. GPT-4o, OpenAI, chatgpt.com, accessed 10:43 on June 30, 2026, prompt: "How do I design the UserProgress model when BA has not provided a spec and why does the 'has not been defined' error keep recurring in Sequelize models?", used to design database schemas and resolve relational errors; AI provided a schema layout (userId, lessonId, lessonType, etc.) and explained cyclic imports/model array registration, student created the migration files and resolved the "has not been defined" error by properly registering all models in the app.module.ts configuration.

**Chat History:** See in Appendix 2.5

## Gia Phúc's AI Usage:

- *Copilot*. Copilot, GitHub via Visual Studio Code, accessed 21:10 on June 21, 2026, prompt: "Where can I edit the size of the whole page?", used to find global frontend styling configurations to modify

layout dimensions; AI identified the global CSS file and relevant structural wrapper selectors, student adjusted the page's container rules in the stylesheet to achieve the correct canvas size.

### Screenshots:

Where can i edit the size of the whole page?

You can change the overall page size from the main wrapper/container in the page component.

For your current roadmap page, the main layout is likely controlled in:

- # RoadmapPage.css

And the page itself is rendered from:

- RoadmapPage.jsx

Common places to edit:

- width: width: ...
- max-width: max-width: ...
- padding: padding: ...
- margin: margin: ...

For example, if you want the whole page to be smaller or larger, look for a container like:

```
.container {
  width: 100%;
  max-width: 1200px;
  margin: 0 auto;
}
```

You can change:

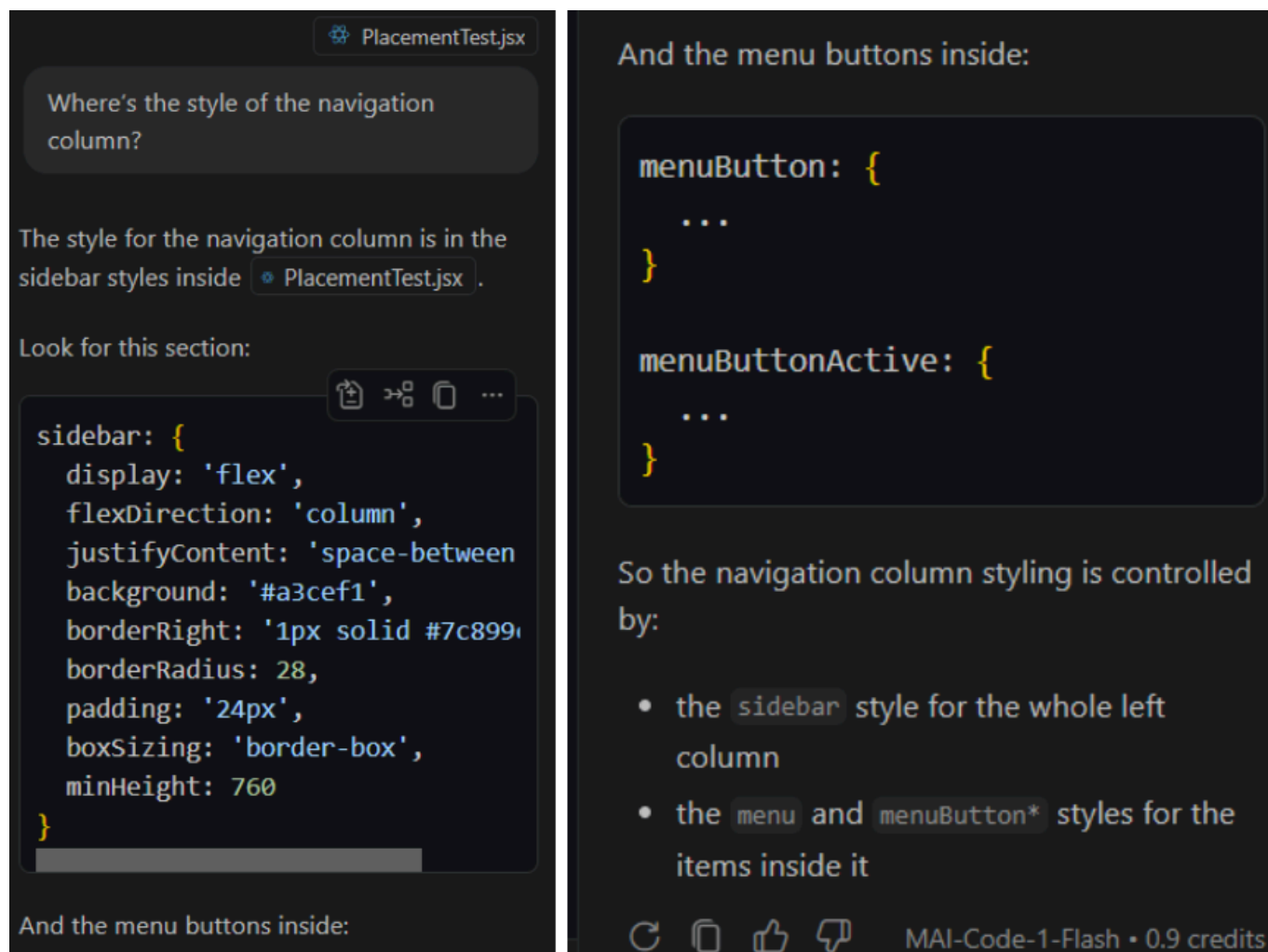
- max-width: 1200px to a smaller/larger value
- padding to add or reduce space around the content

If you want, I can point out the exact container in your roadmap page and tell you which line to edit.

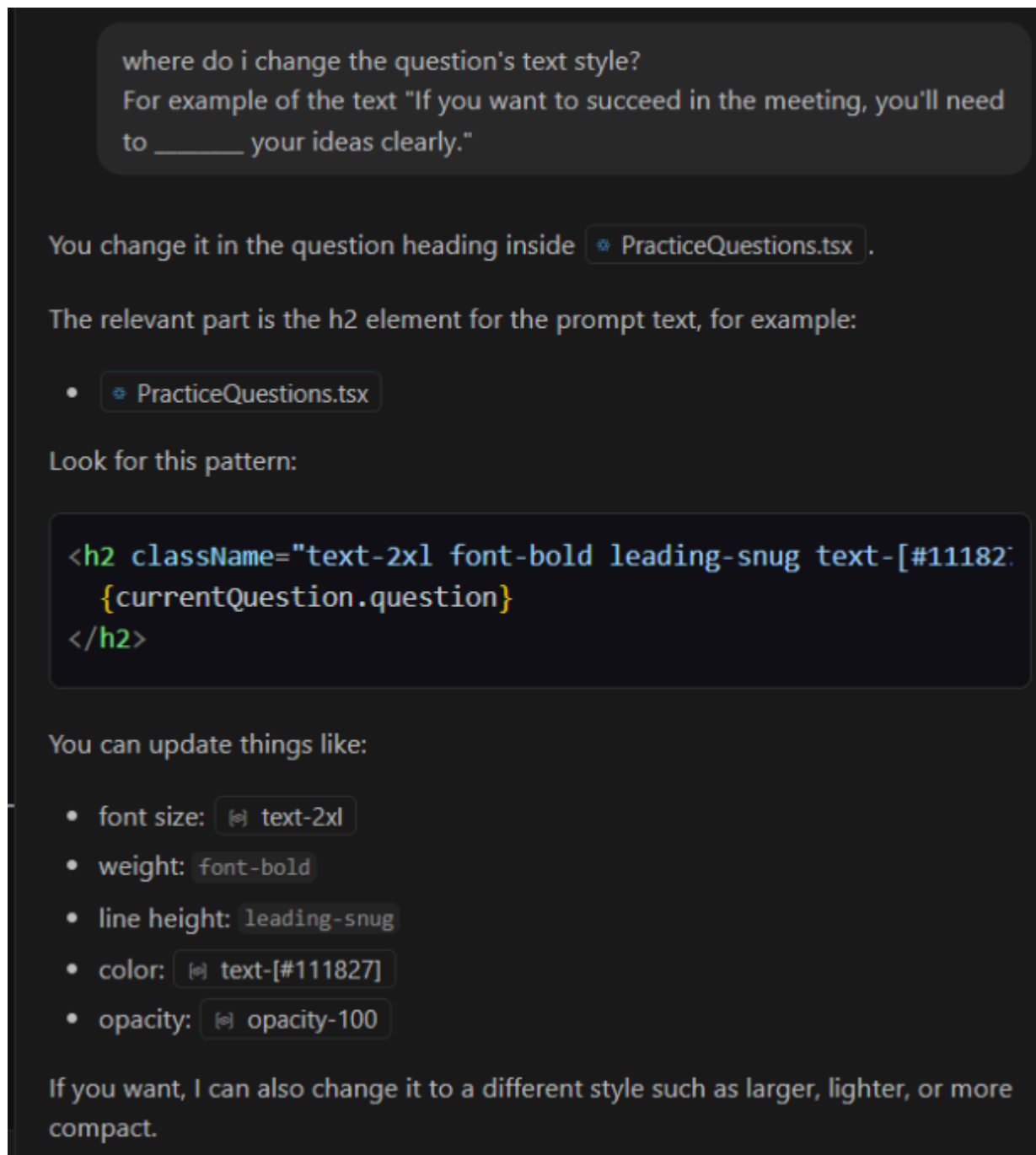
MAI-Code-1-Flash • 0.7 credits

- *Copilot*. Copilot, GitHub via Visual Studio Code, accessed 22:10 on June 24, 2026, prompt: "Where's the style of the navigation column?", used to locate the styling assets for the sidebar/navigation components; AI pointed to the specific layout stylesheet and CSS classes, student modified the column's spacing and structural dimensions to align with the visual design wireframes.

## Screenshots:



- *ChatGPT*. GPT-4o, OpenAI, chatgpt.com, accessed 10:43 on June 27, 2026, prompt: "What is the difference between JWT Guard and Email Verification, is JWT Guard necessary, and what are the two approaches to reset password (6-digit OTP vs reset token)?", used to secure endpoints and choose a reset password architecture; AI compared authentication vs email validity and detailed OTP vs Reset Token logic, student implemented the JWT Guard on all protected routes and chose the 6-digit OTP approach for simple form verification.

**Screenshots:****Khánh Linh's AI Usage:**

- *ChatGPT*. GPT-4o, OpenAI, chatgpt.com, accessed 14:00 on June 15, 2026, prompt: "How do I design a dynamic, hierarchical curriculum tree of Chapters and Lessons in a relational database without encountering N+1 query performance issues?", used to analyze curriculum data structures; AI explained nested-set vs. adjacency list indexing and JSON payload nesting, student independently designed the custom relational response structure and mapped associations for the GET /learning/roadmap endpoint.

**Chat History:** See in Appendix 2.6

- *ChatGPT*. GPT-4o, OpenAI, chatgpt.com, accessed 10:00 on June 17, 2026, prompt: "How to validate a Record<number, string> DTO in NestJS class-validator when receiving non-sequential question IDs from a client payload?", used to build placement test scoring validation; AI detailed

how class-validator handles map-like schemas, student implemented robust criteria parsing inside submit-test.dto.ts for the POST /onboarding/placement-test/submit endpoint.

**Chat History:** See in Appendix 2.7

- *ChatGPT*. GPT-4o, OpenAI, chatgpt.com, accessed 15:30 on June 22, 2026, prompt: "Why does localhost:5173 get blocked by CORS when calling localhost:3000 and how do preflight OPTIONS requests work in NestJS?", used to configure safe cross-origin resource sharing; AI detailed the mechanics of preflight Handshakes and CORS headers, student enabled explicit local rules via app.enableCors() in src/main.ts to unblock frontend integrations.

**Chat History:** See in Appendix 2.8

- *ChatGPT*. GPT-4o, OpenAI, chatgpt.com, accessed 11:15 on June 24, 2026, prompt: "What are the performance differences between loading a massive unified course catalog vs. dynamic, on-demand query retrieval by lessonId?", used to optimize runtime memory consumption; AI compared RAM spikes and frontend asset bundle loading characteristics, student structured the database queries to slice dynamic data retrieval under the standalone GET /learning/lesson-detail API.

**Chat History:** See in Appendix 2.9

- *ChatGPT*. GPT-4o, OpenAI, chatgpt.com, accessed 09:45 on June 26, 2026, prompt: "How to fix NestJS compilation error TS2307: Cannot find module './app.controller' after refactoring module imports?", used to resolve environment build failures; AI identified orphaned controller imports and missing boilerplate references in the root module, student refactored src/app.module.ts manually and restored the application compiling state.

**Chat History:** See in Appendix 2.10

## Thiên Phước's AI Usage:

- Gemini. Gemini 1.5 Pro, Google, gemini.google.com, accessed on July 1, 2026, prompt: "giờ tôi có design trên figma, làm sao để chuyển toàn bộ thành code trong dự án? Phù hợp theo các consitutions của team. Chỉ tôi các bước", used to learn how to convert the team's Figma design into project code while following the team's constitutions; AI generated a general workflow for converting a Figma design into project code, student adapted the guidance to the project context and validated it against the team's implementation rules.

**Chat History:** See in Appendix 2.11

- GitHub Copilot. GPT-5.4 mini, GitHub, Visual Studio Code, accessed on July 1, 2026, prompt: "/mcp.figma. Implement this design from Figma: @https://www.figma.com/design/2NuHveW4S5xEgvbvxrzaeN/Studify?node-id=258-2&m=dev", used to test the most effective workflow for converting the team's Figma design into code files inside the project; AI generated the initial Lesson page implementation, route integration, lesson-card timeline structure, sidebar/header updates, and responsive styling, student reviewed and refined the generated code to match the project structure and validated it with a production build after reinstalling dependencies.

**Chat History:** See in Appendix 2.12

- *Gemini*. Gemini 1.5 Pro, Google, gemini.google.com, accessed on July 2, 2026, prompt: "nhận xét file mình prompt từ MCP Server của Figma theo VSCode? Nó đã đúng với constitutions chưa", used to review whether the Figma MCP prompt and resulting file structure followed the team constitutions; AI generated an assessment of the prompt quality and its alignment with the project constitution, student used the feedback to refine the prompt and verify compliance with the team's implementation rules.

**Chat History:** See in Appendix 2.13

## Minh Thư's AI Usage:

- *Gemini*. Gemini 1.5 Pro, Google, gemini.google.com, accessed 20:39 on July 5, 2026, prompt: "How can I translate a visual roadmap UI design into React/Tailwind CSS, handle SVG namespace properties cleanly within JSX, and safely implement an image asset background without causing grid-tiling duplication bugs?", used to understand responsive dashboard layout architectures, component overlay positioning (relative/absolute tracks), and CSS background scaling mechanisms; AI generated standard responsive boilerplate code and structural troubleshooting fixes, student validated by observing rendering outcomes in local development viewports and manually inserting interaction safety layers (pointer-events-none) to fit the Studify application requirements.

**Chat History:** See in Appendix 2.14

- *ChatGPT*. GPT-4o, OpenAI, chatgpt.com, accessed 20:41 on July 5, 2026, prompt: "How can I refactor a static, hardcoded HTML/CSS signup layout into a fully responsive React TSX component using Tailwind CSS, while preserving the design's structural layout and binding form inputs to controlled React states for backend authentication?", used to adapt static layout mockups into structural, responsive React TSX login and registration forms using Tailwind CSS and controlled component states; AI generated responsive grid bindings and state hook integrations, student validated by checking style scalability on multi-device viewports and reviewing native form submission workflows.

**Chat History:** See in Appendix 2.15

- *ChatGPT*. GPT-4o, OpenAI, chatgpt.com, accessed 22:49 on July 6, 2026, prompt: "How do I dynamically bind active UI navigation states with react-router-dom NavLink components using strict Figma inline styles (specifically background colors, fonts, and borders)?", used to connect React Router hooks to visual design assets; AI generated custom callback styles for , student validated by testing URL parameters and active path matching.

**Chat History:** See in Appendix 2.16

## Appendix: AI Usage Proof

---

### 1

#### 1.1

Stakeholders and users are related concepts, but they are **not** the same.

\* **Stakeholders** are anyone who has an interest in the project's success or is affected by it.

\* **Users** are the people who actually use the system.

---

## # What is a Stakeholder?

A **stakeholder** is any individual, group, or organization that can influence, is affected by, or has an interest in a project.

Stakeholders may:

- \* define requirements,
- \* provide funding,
- \* make decisions,
- \* develop the system,
- \* maintain the system,
- \* or simply be affected by its outcome.

For example, in a university course management system:

| Stakeholder               | Interest                                                     |
|---------------------------|--------------------------------------------------------------|
| -----                     | -----                                                        |
| University administration | Wants the system to support academic management efficiently. |
| Lecturers                 | Need tools to manage courses and grades.                     |
| Students                  | Need an easy way to enroll in courses and view results.      |
| Development team          | Builds and maintains the system.                             |
| IT support                | Operates and maintains the deployed system.                  |

Notice that **students** are both stakeholders and users, while the university administration is a stakeholder but may never use the system directly.

---

## # What is a User?

A **user** is someone who directly interacts with the software to accomplish specific tasks.

Users are often categorized by their roles.



Example for an online learning platform:

| User Type     | Typical Actions                                |
|---------------|------------------------------------------------|
| Student       | Study lessons, take quizzes, track progress    |
| Teacher       | Create courses, manage students                |
| Administrator | Manage users, permissions, and system settings |

Every user is a stakeholder because they are affected by the system, but  
\*\*not every stakeholder is a user\*\*.

---

# How do you identify stakeholders?

A useful approach is to ask:

1. \*\*Who pays for the project?\*\*

- \* Client
- \* Company
- \* Organization

2. \*\*Who decides what features are built?\*\*

- \* Product owner
- \* Project manager
- \* Business analyst

3. \*\*Who develops the system?\*\*

- \* Developers
- \* Designers
- \* QA engineers

4. \*\*Who maintains the system after deployment?\*\*

- \* DevOps
- \* IT support
- \* System administrators

5. \*\*Who is affected by the system?\*\*

- \* Customers
- \* Employees
- \* Students
- \* Patients

Anyone answering "yes" to one of these questions is likely a stakeholder.

---

# How do you identify users?

Ask:

- \* Who will log into the system?
- \* Who performs actions using the software?
- \* Who uses the system to complete daily tasks?

Each distinct group usually becomes a different user type.

For example, in an e-commerce website:

| User          | Main Activities                              |
|---------------|----------------------------------------------|
| Customer      | Browse products, place orders, make payments |
| Seller        | Manage products and orders                   |
| Administrator | Manage users, products, and reports          |

---

# Example: English Learning Platform (e.g., Studify)

### Stakeholders

| Stakeholder                | Reason                                                   |
|----------------------------|----------------------------------------------------------|
| Students                   | Use the platform to learn English and prepare for exams. |
| Teachers                   | Create learning materials and monitor student progress.  |
| Product Owner              | Defines the product vision and priorities.               |
| Development Team           | Designs, builds, tests, and maintains the platform.      |
| University or Organization | May sponsor or adopt the platform.                       |
| System Administrator       | Maintains servers, security, and availability.           |

### Users

| User Type     | Responsibilities                                                         |
|---------------|--------------------------------------------------------------------------|
| Student       | Complete lessons, quizzes, and placement tests; track learning progress. |
| Teacher       | Manage courses, assignments, and student performance.                    |
| Administrator | Manage users, permissions, and platform configuration.                   |

---

## # A simple way to determine stakeholders and users

When analyzing a new project, use this checklist:

1. **Who funds or owns the project?** → Stakeholders
2. **Who makes business decisions?** → Stakeholders
3. **Who builds or maintains the system?** → Stakeholders
4. **Who uses the software directly?** → Users (and therefore also stakeholders)
5. **Can you group users by different goals or permissions?** → These groups become your user types or roles.

By answering these questions, you can systematically identify both the stakeholders and the user roles for almost any software project.

## 2

### 2.1

These are two unrelated concepts, but they're both common when building a backend with NestJS + Sequelize.

---

#### # 1. API-First vs Code-First

The difference is **where the API contract comes from**.

##### ## API-First

In an API-first approach, you design the API **before writing the implementation**.

The workflow typically looks like this:

```
```text
Requirements
  |
  ▼
API Design (OpenAPI / Swagger)
  |
  ▼
Frontend & Backend agree on contract
  |
  ▼
Backend implementation
  |
  ▼
Frontend implementation
```
```

For example, you might first write an OpenAPI specification:

```
```yaml
POST /auth/login

Request:
{
  "email": "string",
  "password": "string"
}

Response:
{
  "accessToken": "string"
}
```
```

Only after everyone agrees on this contract do you start implementing the controller and service.

### ### Advantages

- \* Frontend and backend teams can work in parallel.
- \* Everyone shares a single source of truth.
- \* API changes are reviewed before code is written.
- \* SDKs and client libraries can often be generated automatically.

### ### Disadvantages

- \* More planning is required.
- \* Can feel slower for small projects or prototypes.

---

## ## Code-First

With code-first, you implement the backend first and generate the documentation afterward.

Workflow:

```
```text
Requirements
  |
  ▼
NestJS Controller
  |
  ▼
DTO
  |
  ▼
Swagger decorators
  |
  ▼
```

Generated OpenAPI documentation  
 ```

Example:

```
```ts
@Controller("auth")
export class AuthController {
  @Post("login")
  login(@Body() dto: LoginDto) {
    ...
  }
}
```
```

Using Swagger decorators:

```
```ts
@ApiOperation({ summary: "User login" })
@ApiResponse({ status: 200 })
```
```

NestJS can automatically generate the OpenAPI documentation from your code.

### ### Advantages

- \* Faster development.
- \* Less duplicated work.
- \* Great for small teams.
- \* Documentation stays close to the implementation.

### ### Disadvantages

- \* API design may evolve inconsistently if changes aren't reviewed.
- \* Frontend developers often have to wait until endpoints are implemented.

---

## ## Which one should you use?

| Situation                      | Recommended |
|--------------------------------|-------------|
| -----                          | -----       |
| Startup prototype              | Code-first  |
| University project             | Code-first  |
| Small backend team             | Code-first  |
| Multiple frontend/mobile teams | API-first   |
| Public APIs                    | API-first   |
| Enterprise microservices       | API-first   |

For most NestJS course projects and internal applications, **code-first** is the more practical choice. For large organizations where multiple teams consume the same API, **API-first** provides stronger contract management.

---

## # 2. What is `dataValues` in Sequelize?

Suppose you execute:

```
```ts
const user = await User.findByPk(1);

console.log(user);
```
```

The output might resemble:

```
```text
User {
  dataValues: {
    id: 1,
    name: "Alice",
    email: "alice@example.com"
  },
  _previousDataValues: {
    ...
  },
  _changed: Set(0) {},
  isNewRecord: false
}
```
```

Many developers wonder why their actual data is inside `dataValues`.

---

### ## A Sequelize model is more than plain JSON

The result of `findByPk()` is **a Sequelize model instance**, not a simple JavaScript object.

Internally, it looks conceptually like:

```
```text
User Instance
|
|— dataValues
|— changed()
|— save()
|— destroy()
|— update()
|— validation methods
```
```

The instance contains both:

- \* the row's data, and
- \* methods for interacting with the database.

---

## ## Why `dataValues` exists

Sequelize stores the current column values inside `dataValues`.

Conceptually:

```
```text
User Instance

↓

dataValues

↓

{
  id: 1,
  name: "Alice"
}
```
```

When you access:

```
```ts
user.name
```
```

Sequelize is actually reading:

```
```ts
user.dataValues.name
```
```

through property getters.

---

## ## Why not use `dataValues` directly?

Although this works:

```
```ts
console.log(user.dataValues.name);
```
```

it's generally discouraged.

Instead, use:

```
```ts
console.log(user.name);
```
```

```
```
```

or convert the model into a plain object.

```
---
```

## ## Getting plain JSON

The recommended approach is:

```
```ts
const user = await User.findByPk(1);

const plain = user.toJSON();
```
```

Result:

```
```json
{
  "id": 1,
  "name": "Alice",
  "email": "alice@example.com"
}
```
```

or:

```
```ts
const plain = user.get({ plain: true });
```
```

Both remove Sequelize's internal metadata.

```
---
```

## ## Why are there other internal fields?

You may also see:

```
```text
_previousDataValues
_changed
_options
isNewRecord
```
```

These are used internally by Sequelize.

For example:

```
```ts
user.name = "Bob";
```
```



Sequelize compares:

```
```text
_previousDataValues
```

↓

```
{
  name: "Alice"
}
```
```

with

```
```text
dataValues
```

↓

```
{
  name: "Bob"
}
```
```

to determine which fields changed before generating an `UPDATE` statement.

---

## Example lifecycle

```
```ts
const user = await User.findByPk(1);
```
```

Returns:

```
```text
User Instance
  |
  ▼
dataValues
  |
  ▼
{
  id: 1,
  name: "Alice"
}
```
```

Update the instance:

```
```ts
user.name = "Bob";
```

```
```
```

Then save:

```
```ts
await user.save();
```
```

Sequelize detects that `name` changed and issues an `UPDATE` query only for that field.

```
---
```

## # Best practices

### \* \*\*API design\*\*:

- \* Use **code-first** for most NestJS projects, internal APIs, or student/team projects where the backend drives the implementation.

- \* Use **API-first** when the API is a shared contract across multiple teams or external consumers.

### \* \*\*Sequelize model instances\*\*:

- \* Treat the object returned by `findOne()`, `findByPk()`, or `findAll()` as a **model instance**, not plain JSON.

- \* Access properties through `user.name` rather than `user.dataValues.name`.

- \* When returning data from your API or performing object manipulation, convert the instance with `user.toJSON()` or `user.get({ plain: true })` to avoid exposing Sequelize's internal metadata.

## 2.2

These are two fundamental backend development topics:

1. **How to test APIs**
2. **How JWT authentication works**

They are closely related because after logging in, you'll typically use a JWT to test protected endpoints.

```
---
```

## # Part 1. Ways to test an API

Suppose your NestJS server is running on:

```
```text
http://localhost:3000
```
```

and has an endpoint:

```
```http
POST /auth/login
```
```

There are several ways to call it.

---

### ## Method 1: Terminal (`curl`)

You can send an HTTP request directly from your terminal.

Example:

```
```bash
curl -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "alice@example.com",
    "password": "123456"
  }'
```
```

### ### Pros

- \* Available on most operating systems.
- \* Great for automation and shell scripts.
- \* Useful in CI/CD pipelines.

### ### Cons

- \* Can become difficult to read for complex requests.
- \* No graphical interface.
- \* Harder to work with file uploads or many headers.

---

### ## Method 2: Thunder Client (VS Code)

Thunder Client is a VS Code extension with a graphical interface.

Instead of typing:

```
```bash
curl ...
```
```

you simply:

- \* choose `POST`,
- \* enter the URL,

- \* select **\*\*Body → JSON\*\***,
- \* paste your JSON payload,
- \* click **\*\*Send\*\***.

#### Advantages:

- \* Easy to edit requests.
- \* Stores collections of requests.
- \* Good for day-to-day backend development.
- \* No need to leave VS Code.

---

## ## Method 3: Swagger UI

NestJS integrates well with Swagger.

Example:

```
```ts
const config = new DocumentBuilder()
  .setTitle('API')
  .addBearerAuth()
  .build();
```
```

When you visit:

```
```text
http://localhost:3000/api
```
```

you get interactive documentation.

You can:

- \* read endpoint documentation,
- \* see request and response schemas,
- \* click **\*\*Try it out\*\***,
- \* execute requests directly from the browser.

#### Advantages:

- \* Documentation and testing are combined.
- \* Excellent for sharing APIs with teammates.
- \* Keeps docs synchronized with your code (in a code-first setup).

---

## ## Method 4: Postman (similar to Thunder Client)

Postman provides many advanced features:

- \* environments,

- \* variables,
- \* scripting,
- \* automated API tests,
- \* request collections.

It is widely used for larger projects and team collaboration.

---

## Which should you use?

| Tool           | Best for                                                     |
|----------------|--------------------------------------------------------------|
| -----          | -----                                                        |
| ----           |                                                              |
| `curl`         | Automation, quick checks, scripts                            |
| Thunder Client | Daily development inside VS Code                             |
| Swagger UI     | Interactive documentation and manual testing                 |
| Postman        | Complex workflows, team collaboration, automated API testing |

Many developers use a combination—for example, Swagger while building endpoints, Thunder Client or Postman during development, and `curl` in scripts or documentation.

---

# Part 2. How JWT authentication works

Imagine a user logs into your application.

```

User

↓

POST /auth/login

↓

Server verifies password

↓

Server creates JWT

↓

Server returns token

↓

Client stores token

↓

Client sends token with future requests

```

---

## ## Step 1: Login

Client:

```
```http
POST /auth/login
```
```

Body:

```
```json
{
  "email": "alice@example.com",
  "password": "123456"
}
```
```

---

## ## Step 2: Server verifies credentials

The backend:

- \* finds the user,
- \* compares the password (usually against a hashed password),
- \* rejects invalid credentials.

If successful:

```

User found

↓

Password matches

↓

Generate JWT

```

---

## ## Step 3: Server creates a JWT

The payload might look like:

```
```json
{
  "sub": 15,
  "email": "alice@example.com",
  "role": "student"
}
```
```

The server signs this payload with a secret key.

The resulting token resembles:

```
```text
eyJhbGciOiJIUzI1NiIs...
```
```

---

## Step 4: Client stores the token

Common storage options include:

- \* HttpOnly cookies (recommended for many web applications),
- \* browser memory,
- \* local storage (common but requires careful handling against XSS).

---

## Step 5: Client accesses a protected endpoint

Example request:

```
```http
GET /courses
```
```

Header:

```
```http
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```
```

The word **\*\*Bearer\*\*** tells the server that the following value is the access token.

---

## Step 6: NestJS verifies the token

The authentication guard:

- \* extracts the token,
- \* verifies its signature,
- \* checks expiration,
- \* decodes the payload,
- \* attaches the authenticated user information to the request.

Conceptually:

```

Request

↓

Authorization Header

↓

JWT Guard

↓

Verify Signature

↓

Decode Payload

↓

req.user

↓

Controller

```

---

## JWT structure

A JWT has three Base64URL-encoded parts separated by periods:

```
```text
header.payload.signature
```
```

For example:

```
```text
xxxxx.yyyyy.zzzzz
```
```

### Header



```
```json
{
  "alg": "HS256",
  "typ": "JWT"
}
```
```

Describes how the token is signed.

---

### ### Payload

```
```json
{
  "sub": 15,
  "email": "alice@example.com",
  "role": "student"
}
```
```

Contains claims (information about the user).

**\*\*Important:\*\*** The payload is encoded, **\*\*not encrypted\*\***. Anyone who has the token can decode the header and payload. Therefore, never put sensitive information such as passwords or credit card numbers inside a JWT.

---

### ### Signature

Created using:

```

Header

+

Payload

+

Secret Key

```

The signature allows the server to detect whether the token has been modified.

---

## # Testing JWT-protected APIs

A typical workflow is:

1. Call the login endpoint.

```
```http
POST /auth/login
```
```

2. Copy the returned token.

```
```json
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs..."
}
```
```

3. Send it in subsequent requests.

In Thunder Client or Postman:

```
```
Authorization
```

↓

Bearer Token

↓

```
Paste token
```
```

Or manually add the header:

```
```http
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```
```

In Swagger UI, if you've configured bearer authentication, click **\*\*Authorize\*\***, paste the token once, and Swagger automatically includes the ``Authorization`` header for protected endpoints.

---

# End-to-end authentication flow

```
```text
Client
  |
  | POST /auth/login
  ▼
NestJS
  |
  | Verify credentials
  ▼
Generate JWT
  |
  ▼
```

```

Return access token
  |
  ▼
Client stores token
  |
  GET /courses
  | Authorization: Bearer <token>
  |
  ▼
JWT Guard
  |
  Verify signature & expiration
  |
  ▼
Controller
  |
  ▼
Protected response
  ...

```

This stateless approach is one of the reasons JWT is popular: after login, the server typically doesn't need to keep per-user session state. Each request carries the access token, and the server can independently verify its authenticity and extract the user's identity and permissions before allowing access to protected resources.

## 2.3

This is a common architectural question when building a NestJS application. The confusion usually comes from mixing **authentication** and **user management** responsibilities.

---

# The responsibilities of each module

Think of it like this:

```text

Authentication Module

-----

- ✓ Login
- ✓ Register
- ✓ Logout
- ✓ Refresh Token
- ✓ Forgot Password
- ✓ Verify Email
- ✓ JWT generation
- ✓ Password hashing

User Module

-----

- ✓ Get user profile

- ✓ Update profile
- ✓ Change avatar
- ✓ Change username
- ✓ List users (admin)
- ✓ Delete user
- ✓ User preferences

````

A useful mental model is:

```
> **Auth answers "Who are you?"**
> **User answers "Who is this person?"**
```

---

# Should there be both `/auth/register` and `/user/register`?

Usually, **no**.

A common REST API looks like:

```
```text
POST  /auth/register
POST  /auth/login
POST  /auth/logout

GET    /users/me
PATCH /users/me

GET    /users/:id
DELETE /users/:id
```
```

Registration is part of **authentication**, because creating an account involves:

- \* validating credentials,
- \* hashing the password,
- \* checking duplicate email/username,
- \* creating the account,
- \* optionally issuing a JWT.

Those are authentication concerns.

---

# When might `/users` create users?

There are cases where a separate user creation endpoint makes sense.

For example, an admin dashboard:

```
```text
POST /users
```

```
```
```

could create a new teacher or staff account.

That's different from:

```
```text
POST /auth/register
```
```

where an anonymous visitor signs themselves up.

```
---
```

## # Typical architecture

A clean module separation might look like:

```
```text
src/
├── auth/
│   ├── auth.controller.ts
│   ├── auth.service.ts
│   ├── jwt.strategy.ts
│   └── auth.module.ts
├── users/
│   ├── users.controller.ts
│   ├── users.service.ts
│   └── users.module.ts
```
```

### ### AuthService

Responsible for:

```
```text
Register
```

↓

Hash password

↓

Create user

↓

Generate JWT

↓

Return token

```
```
```

### ### UserService

Responsible for:

```
```text
```

Find user

Update profile

Delete user

Get profile

```
```
```

Notice that `AuthService` often **uses** `UserService` to create or retrieve users rather than duplicating user-related database logic.

---

# Why does registration sometimes call UserService?

A common implementation is:

```
```text
```

AuthController

↓

AuthService.register()

↓

UserService.create()

↓

Database

```
```
```

The authentication layer orchestrates the process, while the user layer owns persistence logic.

---

# Why does register return "Login successfully"?

This is almost certainly a copy-and-paste mistake.

For example:

```
```ts
```

```
async register(dto: RegisterDto) {
```

```

    const user = await ...

    return {
      message: "Login successfully",
      user,
    };
  }
  ...

```

The endpoint works, but the response message wasn't updated.

It should be something like:

```

```ts
return {
  message: "Registration successful",
  user,
};
```

```

or, if you automatically log the user in after registration:

```

```ts
return {
  message: "Registration successful",
  accessToken,
  user,
};
```

```

---

# Should register automatically log the user in?

There are two common patterns.

### Option 1: Register only

```

```text
POST /auth/register

```

↓

Create account

↓

201 Created

↓

```

User manually logs in
```

```

Response:

```
```json
{
  "message": "Registration successful"
}
```
```

---

### Option 2: Register + Auto Login

```
```text
POST /auth/register
```

↓

Create account

↓

Generate JWT

↓

Return token

```
```
```

Response:

```
```json
{
  "message": "Registration successful",
  "accessToken": "...",
  "user": {
    "id": 1,
    "email": "alice@example.com"
  }
}
```
```

This provides a smoother user experience because the user can start using the application immediately.

---

# Should AuthService create users directly?

A common pattern is:

```
```text
AuthService
```

↓



```
UserService.createUser()
```

↓

```
Repository
```

↓

```
Database  
```
```

rather than:

```
```text  
AuthService
```

↓

```
Repository
```

↓

```
Database  
```
```

The advantage is that **all user creation logic lives in one place**.

For example:

```
```text  
UserService.create()
```

↓

```
Validate email
```

↓

```
Validate username
```

↓

```
Save user
```

↓

```
Return user  
```
```

Then `AuthService` focuses on authentication-specific work:

```
```text  
Register
```

↓

Hash password

↓

UserService.create()

↓

Generate JWT

↓

Return response  
```

This keeps responsibilities separated and makes the code easier to maintain.

---

# Recommended endpoint design

For most NestJS applications, a clean API surface looks like this:

| Endpoint              | Responsibility                            |
|-----------------------|-------------------------------------------|
| -----                 | -----                                     |
| -----                 |                                           |
| `POST /auth/register` | Create a new account                      |
| `POST /auth/login`    | Authenticate and return JWT               |
| `POST /auth/refresh`  | Refresh access token                      |
| `POST /auth/logout`   | End the session (if applicable)           |
| `GET /users/me`       | Get the authenticated user's profile      |
| `PATCH /users/me`     | Update the authenticated user's profile   |
| `GET /users/:id`      | Retrieve a user's public or admin-visible |
| information           |                                           |
| `DELETE /users/:id`   | Remove a user (typically admin-only)      |
|                       |                                           |

---

## In your case

If your project currently has both:

```
```text
POST /user/register
POST /auth/register
```
```

ask whether they truly serve different purposes.

- \* If both endpoints perform the **same self-registration flow**, it's better to keep **only `POST /auth/register`** to avoid duplicate logic and confusion.

- \* If `POST /user/register` is intended for **administrative user creation**, give it distinct authorization rules and behavior (for example, only admins can access it).

Finally, if your registration endpoint responds with:

```
```json
{
  "message": "Login successfully"
}
```
```

but it's creating a new account, that's simply an incorrect response message. Update it to reflect the actual operation (e.g., `Registration successful`), and if you're issuing a JWT as part of registration, include the token in the response so the client can authenticate subsequent requests immediately.

## 2.4

These three concepts are related to authentication, but they solve **different security problems**. It's helpful to think about **when** each one is used in the user's lifecycle.



```

---

## # 1. JWT Guard vs Email Verification

These are **not** alternatives—they have different purposes.

### ## JWT Guard

A JWT Guard answers:

> "Is this request coming from an authenticated user?"

Example:

```
```http
GET /users/me
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

The guard:

1. Reads the `Authorization` header.
2. Extracts the JWT.
3. Verifies the signature.
4. Checks whether the token is expired.
5. If valid, attaches the user information to the request.
6. If invalid, returns `401 Unauthorized`.

Without a valid JWT, protected endpoints cannot be accessed.

Typical endpoints protected by a JWT Guard:

```
* `GET /users/me`
* `PATCH /users/me`
* `POST /courses/:id/enroll`
* `POST /quizzes/:id/submit`
```

---

### ## Email Verification

Email verification answers:

> "Does this email address actually belong to the user?"

Typical flow:

```
```text
User registers
  |
  ▼
Account created (verified = false)
```

```

|
▼
Send verification email
|
▼
User clicks verification link
|
▼
verified = true
```
```

This prevents users from registering with fake or mistyped email addresses.

---

## ## Comparison

| JWT Guard                      | Email Verification       |
|--------------------------------|--------------------------|
| -----                          | -----                    |
| Protects APIs                  | Confirms email ownership |
| Used after login               | Used after registration  |
| Checks authentication          | Checks account validity  |
| Runs on each protected request | Usually performed once   |

---

## # 2. Is JWT Guard necessary?

For **public** endpoints, no.

Examples:

```
```text
POST /auth/login
POST /auth/register
GET /courses/public
```
```

Anyone should be able to call these.

---

For **private** endpoints, yes.

Imagine:

```
```http
PATCH /users/me
```
```

Without authentication, anyone could attempt to modify user data.

With a JWT Guard:

```
```text
Request
  |
  ▼
JWT Guard
  |
  ├── Valid token → Continue
  └── Invalid token → 401 Unauthorized
```
```

In most applications, a JWT Guard is the standard way to protect authenticated routes.

---

### # 3. Password reset approaches

There are two common designs.

#### ## Option 1: 6-digit OTP (One-Time Password)

Flow:

```
```text
Forgot Password
  |
  ▼
Enter email
  |
  ▼
Server generates:
483921
  |
  ▼
Email the code
  |
  ▼
User enters the code
  |
  ▼
Server verifies it
  |
  ▼
Allow password change
```
```

Example:

```
```
Your verification code is:

483921

Expires in 10 minutes.
```
```

```
```
```

### ### Advantages

- \* Simple user experience.
- \* Works well on mobile devices.
- \* Familiar from banking and messaging apps.

### ### Considerations

- \* Store the code securely (preferably hashed).
- \* Set a short expiration time.
- \* Limit retry attempts to reduce brute-force attacks.

```
---
```

## ## Option 2: Reset Token (Email Link)

Flow:

```
```text
```

Forgot Password



Enter email



Generate random token



Email a reset link



User clicks the link



Server validates token



Allow password change

```
```
```

Example email:

```
```text
```

<https://example.com/reset-password?token=abc123xyz...>

```
```
```

### ### Advantages

- \* Very common for web applications.
- \* Users don't need to copy a code manually.
- \* Easy to integrate with a web frontend.

### ### Considerations

- \* Tokens should be long, cryptographically random, and single-use.
- \* They should expire after a short period (e.g., 15–30 minutes).
- \* Store only a hash of the token if persisting it in the database.

---

#### # Which approach is better?

| Feature                        | 6-digit OTP | Reset Token |
|--------------------------------|-------------|-------------|
| User enters code manually      | ✓           | ✗           |
| User clicks a link             | ✗           | ✓           |
| Common for mobile apps         | ✓           | Sometimes   |
| Common for web apps            | Less common | ✓           |
| Easier to automate in browsers | ✗           | ✓           |

Neither is inherently more secure—the security comes from:

- \* short expiration times,
- \* single-use credentials,
- \* rate limiting,
- \* secure random generation,
- \* and HTTPS.

---

#### # Can they be combined?

Yes.

Some systems send a link that opens a page where the user must also enter a short verification code. This provides an additional layer of assurance, although it's usually unnecessary for standard consumer applications.

---

#### # Recommended architecture for a typical NestJS application





```

|
▼
Forgot Password
|
├── Option A: Email 6-digit OTP
|
└── Option B: Email reset link with secure token
...

```

## ## Practical recommendations

For most modern web applications built with NestJS:

- \* Use **JWT Guards** to protect authenticated API endpoints.
- \* Use **email verification** if users sign up with email addresses and account ownership matters.
- \* For password resets:

- \* **Web applications** typically provide a **password reset link** containing a secure, single-use token because it's convenient and familiar.

- \* **Mobile applications** often favor a **6-digit OTP** because entering a short code is a natural part of the mobile experience.

The important point is that these features complement each other rather than replace one another: email verification establishes trust in the account, JWT Guards protect authenticated resources, and password reset flows provide a secure recovery mechanism if the user loses access to their password.

## 2.5

These are actually two different problems:

1. **A system design problem:** \*How do I design `UserProgress` when the Business Analyst (BA) hasn't specified the requirements?\*
2. **A Sequelize configuration problem:** \*Why do I keep getting `"Model X has not been defined"`?\*

---

### # Part 1. Designing `UserProgress` without a complete specification

The first thing to remember is:

> **Don't invent business rules. Model only what you know.**

If the BA hasn't specified how progress should work, avoid designing an overly complex schema that might need to be rewritten later.

---

## Start from the user stories

Ask questions like:

- \* What does "progress" actually mean?
- \* Is it tracked per course?
- \* Per chapter?
- \* Per lesson?
- \* Per video?
- \* Per quiz?
- \* Can a user revisit lessons?
- \* Is there a completion percentage?
- \* Is "completed" enough?

Without answers, keep the design minimal.

---

## Option 1: Progress per lesson (most common)

```text

User

-----

id

Lesson

-----

id

UserProgress

-----

userId

lessonId

status

completedAt

updatedAt

```

Example:

| userId | lessonId | status      |
|--------|----------|-------------|
| -----: | -----:   | -----       |
| 1      | 101      | COMPLETED   |
| 1      | 102      | IN_PROGRESS |
| 2      | 101      | COMPLETED   |

Advantages:

- \* Simple
- \* Easy to compute course progress
- \* Works well for most LMS applications

Course progress can then be calculated dynamically:

```
```text
Completed lessons
-----
```

```
15
```

```
Total lessons
-----
```

```
20
```

```
Progress = 75%
```
```

No need to store the percentage unless performance becomes a concern.

```
---
```

## ## Option 2: Store video position

If lessons contain videos:

```
```text
UserProgress
-----
userId
lessonId
lastPositionSeconds
completed
```
```

Example:

|        |              |
|--------|--------------|
| lesson | lastPosition |
| -----  | -----        |
| 101    | 245          |

When the learner returns:

```
```text
Resume from
```

```
04:05
```
```

```
---
```

## ## Option 3: Rich progress tracking

Some platforms need more information.

```
```text
UserProgress
-----
userId
```

```

lessonId
status
startedAt
completedAt
lastVisitedAt
timeSpent
attemptCount
```

```

This supports analytics and learning dashboards but requires clear business requirements.

---

## Recommendation when requirements are unclear

Start with the smallest model that supports the current functionality:

```

```text
UserProgress
-----
userId
lessonId
status
completedAt
```

```

As new requirements arrive (resume videos, time tracking, quiz attempts), extend the model instead of guessing now.

---

# Part 2. Why "has not been defined" keeps occurring in Sequelize

An error such as:

```

```text
UserProgress has not been defined
```

```

or

```

```text
Lesson has not been defined
```

```

usually means Sequelize cannot find the model when associations are being created.

---

## Common cause 1: Model not registered

Suppose you have:

```
```ts
sequelize.addModels([
  User,
  Course,
  Lesson,
]);
```
```

but forget:

```
```ts
UserProgress
```
```

When Sequelize encounters:

```
```ts
User.hasMany(UserProgress);
```
```

it doesn't know about `UserProgress` yet.

Make sure every model is registered with Sequelize (or included in the module configuration if using `sequelize-typescript`).

---

## Common cause 2: Circular imports

Example:

```
```text
User.ts
```

↓

```
imports UserProgress
```

↓

```
UserProgress.ts
```

↓

```
imports User
```
```

This cycle can leave one of the classes undefined during initialization.

A common way to avoid this in `sequelize-typescript` is to use callback references:

```
```ts
```

```
@BelongsTo(() => User)
user: User;
```
```

instead of directly referencing the class during module evaluation.

---

### ## Common cause 3: Association defined too early

If you write:

```
```ts
User.hasMany(UserProgress);
```
```

before `UserProgress` has been initialized or registered, Sequelize will throw a "not defined" error.

Ensure all models are loaded before associations are established.

---

### ## Common cause 4: Incorrect import

For example:

```
```ts
import UserProgress from './user-progress.model';
```
```

while the model is exported as:

```
```ts
export class UserProgress extends Model {}
```
```

The correct import is:

```
```ts
import { UserProgress } from './user-progress.model';
```
```

The export style and import style must match.

---

### ## Common cause 5: Wrong model name

Sometimes developers refer to:

```
```ts
sequelize.models.UserProgress
```
```

but the model was registered under a different name.

Verify that the model class name, registration, and association references are consistent.

---

# Typical association setup

```
```text
User
  |
  | 1:N
  ▼
UserProgress
  ▲
  | N:1
Lesson
```
```

Example with decorators:

```
```ts
@Table
export class UserProgress extends Model {
  @ForeignKey(() => User)
  @Column
  userId: number;

  @ForeignKey(() => Lesson)
  @Column
  lessonId: number;

  @BelongsTo(() => User)
  user: User;

  @BelongsTo(() => Lesson)
  lesson: Lesson;
}
```
```

And the related models define the inverse associations:

```
```ts
@HasMany(() => UserProgress)
progress: UserProgress[];
```
```

---

# Debugging checklist

When you see:

```
```text
Model X has not been defined
```
```

check the following:

1. Is the model actually exported?
2. Is it imported correctly (named vs default export)?
3. Is it registered with Sequelize?
4. Are all associations declared after the model is available?
5. Is there a circular import?
6. Are you referencing the correct model class?

---

# Recommendation

### For `UserProgress`

Until the BA provides more detailed requirements, model **only** the data required by the current features. A table keyed by `(userId, lessonId)` with a status (and optionally `completedAt`) is a solid starting point. Derive aggregate values like course completion percentage from lesson progress rather than storing them prematurely.

### For the recurring Sequelize error

Treat `"has not been defined"` as an **application initialization problem**, not a database problem. In most cases, the cause is one of:

- \* the model wasn't registered,
- \* the import/export doesn't match,
- \* there's a circular dependency between models,
- \* or associations are being resolved before all models have been loaded.

Checking those areas systematically will resolve the vast majority of these errors.

## 2.6

When you need to model a curriculum like this:

```
```text
Course
├── Chapter 1
│   ├── Lesson 1
│   ├── Lesson 2
│   └── Lesson 3
└── Chapter 2
    └── Lesson 4
```
```



```

└─ Lesson 5
└─ Chapter 3
   └─ Lesson 6
   └─ Lesson 7
...

```

there are actually **two separate problems**:

1. **How should the database be designed?**
2. **How can the API fetch everything efficiently without N+1 queries?**

Many beginners mix these together, but solving them separately leads to a much cleaner architecture.

---

# Option 1 (Recommended): Separate Chapter and Lesson tables

This is the design used by most LMS systems (Udemy, Coursera, Moodle, Canvas, etc.).

## Database schema

```

```text
Course
-----
id
title
description

Chapter
-----
id
course_id
title
position

Lesson
-----
id
chapter_id
title
content
video_url
duration
position
```

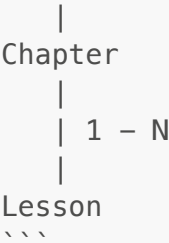
```

Relationships:

```

```text
Course
  |
  | 1 - N

```



Example data

### Course

| id | title        |
|----|--------------|
| 1  | React Basics |

### Chapter

| id | course_id | title        | position |
|----|-----------|--------------|----------|
| 1  | 1         | Introduction | 1        |
| 2  | 1         | Components   | 2        |
| 3  | 1         | Hooks        | 3        |

### Lesson

| id | chapter_id | title                 | position |
|----|------------|-----------------------|----------|
| 1  | 1          | Welcome               | 1        |
| 2  | 1          | Installation          | 2        |
| 3  | 2          | Functional Components | 1        |
| 4  | 2          | Props                 | 2        |
| 5  | 3          | useState              | 1        |

# Why this design is good

Because

- \* chapters have chapter-specific fields
- \* lessons have lesson-specific fields

For example

```
```sql
Chapter
-----
estimated_time
```

Lesson

-----

video\_url  
quiz\_id  
markdown\_content  
```

Everything stays normalized.

---

# The N+1 Problem

Suppose you do this.

```

Get Course

↓

Get Chapters

↓

For every chapter

↓

Get Lessons

```

SQL becomes

```sql  
SELECT \* FROM chapter WHERE course\_id=1;  
```

returns

```

3 chapters

```

then

```sql  
SELECT \* FROM lesson WHERE chapter\_id=1;  
```

then

```sql  
SELECT \* FROM lesson WHERE chapter\_id=2;  
```

then

```
```sql
SELECT * FROM lesson WHERE chapter_id=3;
```
```

Total:

```
```
1 + N queries
```
```

If

```
```
100 chapters
```
```

then

```
```
101 SQL queries
```
```

This is the classic **\*\*N+1 problem\*\***.

---

# Solution 1: Fetch everything at once

Instead of

```
```
Chapter
↓
```

Lesson

↓

Lesson

↓

```
Lesson
```
```

fetch

```
```sql
SELECT *
FROM Chapter
LEFT JOIN Lesson
```

```
ON Chapter.id = Lesson.chapter_id
WHERE Chapter.course_id = 1
ORDER BY Chapter.position,
         Lesson.position;
...
```

Only **one SQL query**.

The backend groups the rows.

Result

```
```json
[
  {
    "chapterId":1,
    "title":"Introduction",
    "lessons":[
      {"id":1},
      {"id":2}
    ]
  },
  {
    "chapterId":2,
    "title":"Components",
    "lessons":[
      {"id":3},
      {"id":4}
    ]
  }
]
```
```

---

# Solution 2: Two-query strategy (often even better)

Many ORMs (like Prisma, TypeORM, Hibernate) can produce duplicate chapter data when using joins. A common pattern is:

Query 1:

```
```sql
SELECT *
FROM Chapter
WHERE course_id = 1
ORDER BY position;
```
```

Query 2:

```
```sql
SELECT *
FROM Lesson
```

```
WHERE chapter_id IN (1,2,3);
```

```

Then group in memory.

Only

```
```

```

2 queries

```
```

```

instead of

```
```

```

101 queries.

```
```

```

This approach scales well and avoids repeating chapter columns for every lesson.

```
---
```

### # Solution 3: ORM eager loading

For example, in Prisma:

```
```ts
const course = await prisma.course.findUnique({
  where: { id: courseId },
  include: {
    chapters: {
      orderBy: {
        position: 'asc'
      },
      include: {
        lessons: {
          orderBy: {
            position: 'asc'
          }
        }
      }
    }
  }
});
```

```

Or in TypeORM:

```
```ts
courseRepository.find({
  relations: {
    chapters: {
      lessons: true
    }
  }
});
```

```

```
}  
});  
```
```

The ORM batches the necessary SQL instead of issuing one query per chapter.

---

# Option 2: One "Node" table (Tree structure)

Some systems need arbitrary nesting, such as:

```
```  
Module  
  |  
Section  
  |  
Subsection  
  |  
Lesson  
```
```

In that case:

```
```text  
CurriculumNode  
-----  
id  
course_id  
parent_id  
type  
title  
position  
```
```

Example

| id | parent | type    | title      |
|----|--------|---------|------------|
| -- | -----  | -----   | -----      |
| 1  | NULL   | chapter | Intro      |
| 2  | 1      | lesson  | Welcome    |
| 3  | 1      | lesson  | Install    |
| 4  | NULL   | chapter | Components |
| 5  | 4      | lesson  | Props      |

This is known as an **\*\*adjacency list\*\***.

Advantages:

- \* unlimited hierarchy
- \* flexible

Disadvantages:

- \* recursive queries
- \* harder validation
- \* more complicated API
- \* overkill for a simple course → chapter → lesson structure

---

## # API response

Even if the database is normalized, the frontend typically receives nested JSON:

```
```json
{
  "id": 1,
  "title": "React Basics",
  "chapters": [
    {
      "id": 1,
      "title": "Introduction",
      "lessons": [
        {
          "id": 1,
          "title": "Welcome"
        },
        {
          "id": 2,
          "title": "Installation"
        }
      ]
    },
    {
      "id": 2,
      "title": "Components",
      "lessons": [
        {
          "id": 3,
          "title": "Props"
        }
      ]
    }
  ]
}
```
```

The frontend doesn't need to know whether the backend used joins, eager loading, or batched queries.

---

## # Best practice for most LMS projects

For a standard learning platform with a fixed hierarchy (\*\*Course → Chapter → Lesson\*\*), the following approach is typically the best balance of



simplicity and performance:

- \* Use separate `Course`, `Chapter`, and `Lesson` tables with foreign keys.
- \* Add a `position` column to `Chapter` and `Lesson` for ordering instead of relying on IDs.
- \* Fetch the hierarchy using either:
  - \* a single `JOIN` query when the dataset is modest, or
  - \* the efficient **two-query strategy** (all chapters, then all lessons with `WHERE chapter\_id IN (...)`) for larger datasets.
- \* Build the nested JSON structure in the service layer before returning it to the frontend.
- \* Avoid querying lessons inside a loop over chapters, as this causes the N+1 query problem.

This design is simple to maintain, keeps the schema normalized, works well with most ORMs, and scales efficiently as courses grow in size.

## 2.7

When working with NestJS, a `Record<number, string>` is a little tricky because **TypeScript's `number` keys become strings in JSON**. For example, the client sends:

```
```json
{
  "answers": {
    "101": "A",
    "205": "C",
    "999": "B"
  }
}
```
```

Although your TypeScript type may be:

```
```ts
Record<number, string>
```
```

at runtime it is actually:

```
```ts
Record<string, string>
```
```

because JSON object keys are always strings.

---

**## Option 1 (Recommended):** Validate the object, then validate keys in a

custom validator

DTO:

```
```ts
import { IsObject } from 'class-validator';

export class SubmitAnswersDto {
  @IsObject()
  answers: Record<string, string>;
}
```
```

Then create a custom validator.

```
```ts
import {
  ValidatorConstraint,
  ValidatorConstraintInterface,
} from 'class-validator';

@ValidatorConstraint({ name: 'ValidAnswerMap', async: false })
export class ValidAnswerMapConstraint
  implements ValidatorConstraintInterface
{
  validate(value: Record<string, string>) {
    if (!value || typeof value !== 'object') {
      return false;
    }

    return Object.entries(value).every(([key, answer]) => {
      // key must be numeric
      if (!/^\d+$/.test(key)) {
        return false;
      }

      // value must be non-empty
      return typeof answer === 'string' && answer.trim().length > 0;
    });
  }

  defaultMessage() {
    return 'answers must be a map of numeric question IDs to non-empty answers';
  }
}
```
```

Use it:

```
```ts
import { Validate } from 'class-validator';

export class SubmitAnswersDto {
```

```
@Validate(ValidAnswerMapConstraint)
answers: Record<string, string>;
}
```
```

This works well when question IDs are dynamic and non-sequential.

---

## Option 2: Validate against existing question IDs (Business validation)

Suppose your database contains:

```
```
Question IDs

101
205
999
1500
```
```

A client sends:

```
```json
{
  "answers": {
    "101": "A",
    "205": "C",
    "7777": "D"
  }
}
```
```

Structurally, this is valid JSON, but `7777` is not a valid question.

This kind of validation belongs in the **service layer**, not in `class-validator`, because it requires a database lookup.

Example:

```
```ts
const questionIds = Object.keys(dto.answers).map(Number);

const existingQuestions = await
this.questionRepository.findByIds(questionIds);

if (existingQuestions.length !== questionIds.length) {
  throw new BadRequestException('Some question IDs do not exist.');
```

This keeps DTO validation focused on structure and the service responsible for domain rules.

---

### ## Option 3: Transform keys into numbers

If your application prefers numeric IDs after validation, you can transform the keys:

```
```ts
const answers = Object.fromEntries(
  Object.entries(dto.answers).map(([key, value]) => [
    Number(key),
    value,
  ]),
);
```
```

After transformation:

```
```ts
{
  101: "A",
  205: "C",
  999: "B"
}
```
```

Even though JavaScript still stores object property names internally as strings, you'll work with numeric IDs in your application logic.

---

### ## Option 4: Use an array instead of a record (often preferable)

Instead of:

```
```json
{
  "answers": {
    "101": "A",
    "205": "B"
  }
}
```
```

use:

```
```json
{
  "answers": [
    {
      "questionId": 101,
      "answer": "A"
    },
  ],
}
```

```

    {
      "questionId": 205,
      "answer": "B"
    }
  ]
}
...

```

DT0:

```

```ts
import {
  IsInt,
  IsString,
  ValidateNested,
} from 'class-validator';
import { Type } from 'class-transformer';

class AnswerDto {
  @IsInt()
  questionId: number;

  @IsString()
  answer: string;
}

export class SubmitAnswersDto {
  @ValidateNested({ each: true })
  @Type(() => AnswerDto)
  answers: AnswerDto[];
}
...

```

### Why many APIs prefer this

An array of objects is easier to:

- \* validate with built-in decorators,
- \* extend later (e.g., add `timeSpent`, `isFlagged`, `confidence`),
- \* document in Swagger/OpenAPI,
- \* map to database entities.

For example, you can later evolve the schema without breaking clients:

```

```json
{
  "questionId": 101,
  "answer": "A",
  "timeSpent": 12,
  "confidence": 0.9
}
...

```

---

## Which approach should you choose?

| Scenario design                                                                                                                                                                                                       | Recommended  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| -----                                                                                                                                                                                                                 | -----        |
| Simple lookup of question ID → answer<br>`Record<string, string>` with a custom validator                                                                                                                             |              |
| Need to verify IDs exist in the database<br>structure in the DTO, then perform database validation in the service  <br>API expected to evolve with additional per-answer fields   `AnswerDto[]`<br>(array of objects) | Validate<br> |

For a production NestJS application, a common pattern is:

- \* **DTO:** Validate that the payload is a well-formed object or array.
- \* **Custom validator:** Ensure keys (or `questionId`s) have the expected format.
- \* **Service layer:** Verify the referenced question IDs exist and enforce business rules (e.g., no duplicate answers, all required questions answered, quiz is still open). This separation keeps validation responsibilities clear and makes the codebase easier to maintain.

## 2.8

This is one of the most common issues when developing a React (Vite) frontend with a NestJS backend.

Even though both applications are running on your own machine:

- \* Frontend: `http://localhost:5173`
- \* Backend: `http://localhost:3000`

the browser considers them **different origins** because the **port numbers differ**.

An **origin** is defined by:

```

` ` `
protocol + hostname + port
` ` `

```

So:

| URL                     | Origin                  |
|-------------------------|-------------------------|
| -----                   | -----                   |
| `http://localhost:5173` | `http://localhost:5173` |
| `http://localhost:3000` | `http://localhost:3000` |

Since the ports are different, the browser enforces the **Same-Origin Policy**, which prevents JavaScript from freely accessing resources from another origin unless the server explicitly allows it through **CORS** (Cross-Origin Resource Sharing).

---

## # What actually happens

Suppose your React app executes:

```
``ts
await axios.post(
  "http://localhost:3000/auth/login",
  {
    email,
    password,
  }
);
``
```

You might imagine the browser simply sends:

```
``
POST /auth/login
``
```

but that's not always what happens.

For many cross-origin requests, the browser first sends a **preflight request**.

``

React

↓

OPTIONS /auth/login

↓

NestJS

↓

200 OK

Access-Control-Allow-Origin

Access-Control-Allow-Methods

Access-Control-Allow-Headers

↓

Browser approves

↓

POST /auth/login

↓

Response

```

The initial `OPTIONS` request is called the **preflight** request.

---

# Why is a preflight request needed?

Simple cross-origin requests (for example, a plain `GET`) usually don't require one.

However, requests often trigger a preflight if they:

- \* use methods such as `PUT`, `PATCH`, or `DELETE`,
- \* include custom headers (e.g., `Authorization`),
- \* use a content type like `application/json`.

For example:

```
```ts
axios.post(
  "/login",
  data,
  {
    headers: {
      Authorization: "...",
    },
  },
);
```
```

The browser first asks the server:

> "If I send this POST request with these headers, will you allow it?"

That's the purpose of the `OPTIONS` request.

---

# What does the preflight request look like?

The browser sends something similar to:

```
```http
OPTIONS /auth/login HTTP/1.1

Origin: http://localhost:5173
```



Access-Control-Request-Method: POST

Access-Control-Request-Headers: Content-Type, Authorization  
 \ \ \

Notice that **this request is generated automatically by the browser**—your React code never calls ``OPTIONS`` directly.

---

# What should NestJS return?

If CORS is enabled correctly, the response includes headers such as:

\ \ \ http

HTTP/1.1 204 No Content

Access-Control-Allow-Origin: http://localhost:5173

Access-Control-Allow-Methods: GET,POST,PATCH,DELETE

Access-Control-Allow-Headers: Content-Type, Authorization  
 \ \ \

Once the browser sees these headers, it proceeds with the real ``POST``.

---

# What happens if CORS is not configured?

The flow becomes:

\ \ \

React

↓

OPTIONS

↓

NestJS

↓

404

\ \ \

or

\ \ \

OPTIONS

↓

200

(no Access-Control-Allow-Origin header)  
```

In either case, the browser blocks the request before your controller is reached.

You'll typically see an error like:

```
```text
Access to XMLHttpRequest at
'http://localhost:3000/auth/login'
from origin
'http://localhost:5173'
has been blocked by CORS policy.
```
```

This is **a browser security restriction**, not a NestJS exception.

---

## # Enabling CORS in NestJS

The standard solution is to enable CORS when bootstrapping the application.

```
```ts
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.enableCors({
    origin: 'http://localhost:5173',
    credentials: true,
    methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
    allowedHeaders: ['Content-Type', 'Authorization'],
  });

  await app.listen(3000);
}

bootstrap();
```
```

With this configuration, NestJS responds appropriately to both the preflight request and the subsequent API request.

---

## # What about credentials?

If you're using cookies or session-based authentication:

Frontend:

```
``ts
axios.defaults.withCredentials = true;
```
```

Backend:

```
``ts
app.enableCors({
  origin: 'http://localhost:5173',
  credentials: true,
});
```
```

In this case, you **cannot** use:

```
``ts
origin: '*'
```
```

because browsers reject wildcard origins when credentials are included.

---

# Common misconceptions

### "I'm calling localhost, so why is it cross-origin?"

The browser doesn't care that both applications are on your computer.

```

localhost:5173

≠

localhost:3000

```

Different ports mean different origins.

---

### "My API works in Postman but not in Chrome."

Postman is an HTTP client.

Browsers enforce the Same-Origin Policy and CORS.

Postman does not.

That's why the same request may succeed in Postman while being blocked in the browser.

---

### "My NestJS controller never gets called."

Often the request never reaches your controller because the browser rejects the **preflight `OPTIONS` response** before sending the actual **`POST`**.

You can verify this in your browser's Developer Tools → **Network** tab. If you see an **`OPTIONS`** request failing, focus on your CORS configuration rather than your controller logic.

---

# Typical development architecture

```
```text
Browser
  |
  ▼
React (localhost:5173)
  |
  | HTTP
  ▼
NestJS (localhost:3000)
  |
  ▼
Database
```
```

The browser acts as the security gatekeeper. Before forwarding cross-origin requests from the React app to the NestJS server, it checks whether the server explicitly permits them through CORS headers. If the preflight check fails, the actual API request is never sent.

## 2.9

This is a classic **data loading strategy** question. The trade-off is not just about database performance—it also affects **network usage, frontend rendering, memory consumption, caching, and user experience**.

Let's compare the two approaches.

---

# Option 1: Load the entire course catalog

The client requests:

```
```http
GET /courses/1
```
```

The backend returns:

```
```json
{
  "id": 1,
  "title": "Data Structures",
  "chapters": [
    {
      "id": 1,
      "title": "Arrays",
      "lessons": [
        {
          "id": 1,
          "title": "...",
          "content": "...",
          "video": "...",
          "quiz": [...]
        }
      ]
    }
  ]
}
```

Everything needed for the course is transferred in one response.

## ## Advantages

- \* Only one API request.
- \* Navigation between lessons is instantaneous.
- \* Simple frontend logic—no additional loading states when changing lessons.
- \* Easy to cache the entire course.

## ## Disadvantages

Imagine a large course with:

- \* 80 chapters
- \* 1,200 lessons
- \* Markdown content
- \* Video metadata
- \* Quizzes
- \* Attachments
- \* Progress data

The payload can easily reach several megabytes.

This leads to:

- \* slower initial page load,

- \* increased memory usage in the browser,
- \* higher bandwidth consumption,
- \* downloading lesson content the user may never view.

---

# Option 2: Load metadata first, then fetch lessons on demand

The initial request returns only the course structure.

```
```http
GET /courses/1
```
```

Response:

```
```json
{
  "id": 1,
  "title": "Data Structures",
  "chapters": [
    {
      "id": 1,
      "title": "Arrays",
      "lessons": [
        {
          "id": 101,
          "title": "Introduction"
        },
        {
          "id": 102,
          "title": "Traversal"
        }
      ]
    }
  ]
}
```

When the learner opens Lesson 101:

```
```http
GET /lessons/101
```
```

The backend returns:

```
```json
{
  "id": 101,
  "content": "...",
  "video": "...",
  "quiz": [...]
}
```

```
```
```

Only the selected lesson is transferred.

```
---
```

### # Performance comparison

| Aspect              | Entire Course | On-Demand (`lessonId`) |
|---------------------|---------------|------------------------|
| Initial payload     | Large         | Small                  |
| First page load     | Slower        | Faster                 |
| Number of API calls | Few           | More                   |
| Lesson switching    | Instant       | May require a request  |
| Browser memory      | High          | Low                    |
| Bandwidth usage     | Higher        | Lower                  |
| Scalability         | Moderate      | High                   |

```
---
```

### # Database performance

From the database's perspective:

#### ### Entire course

```
```sql
SELECT *
FROM chapters
JOIN lessons
JOIN videos
JOIN quizzes
WHERE course_id = 1;
```
```

This may involve several joins and return hundreds or thousands of rows.

#### Pros:

- \* One database round trip.
- \* Efficient for small datasets.

#### Cons:

- \* Heavy queries.
- \* More data transferred from the database.
- \* Repeated chapter information due to joins.

```
---
```

#### ### On-demand

Initial query:

```
```sql
SELECT id, title
FROM lessons
WHERE course_id = 1;
```
```

Later:

```
```sql
SELECT *
FROM lessons
WHERE id = 101;
```
```

Each query is lightweight and indexed by the lesson's primary key.

---

## # User experience

Consider two learners.

### ### Learner A

- \* Opens the course.
- \* Reads only Lesson 1.
- \* Closes the browser.

If the application loaded all 500 lessons upfront, most of the downloaded data was unnecessary.

---

### ### Learner B

- \* Studies every lesson sequentially.

In this case, repeatedly requesting each lesson introduces extra network round trips.

A good frontend cache can mitigate this by keeping previously loaded lessons in memory.

---

## # Hybrid approach (commonly used)

Most production Learning Management Systems adopt a hybrid strategy.

### ### Step 1

Load the course outline.

```
```text
```



```
Course
├── Chapter 1
│   ├── Lesson 101
│   └── Lesson 102
└── Chapter 2
    └── Lesson 201
```
```

The client immediately displays the navigation.

---

### ### Step 2

When Lesson 101 is selected:

```
```http
GET /lessons/101
```
```

Only that lesson's detailed content is returned.

---

### ### Step 3 (Optimization)

After Lesson 101 loads, the frontend can quietly prefetch Lesson 102.

```
```text
User opens Lesson 101
    |
    ▼
Background fetch Lesson 102
    |
    ▼
Next click feels almost instant
```
```

Libraries such as TanStack Query (React Query) or SWR make this pattern straightforward by handling caching and background fetching automatically.

---

### # Caching considerations

Loading on demand becomes much more efficient with caching:

- \* Cache lesson data by `lessonId`.
- \* Reuse cached lessons when users navigate back.
- \* Configure a cache expiration time if lesson content changes infrequently.

This avoids repeated database and network requests while keeping the initial payload small.

---

## # Recommendation

For most modern LMS applications:

### \* \*\*Load upfront:\*\*

- \* Course metadata
- \* Chapter hierarchy
- \* Lesson IDs, titles, durations, and progress summaries

### \* \*\*Load on demand:\*\*

- \* Lesson body/content
- \* Videos or streaming URLs
- \* Quizzes and question sets
- \* Attachments and downloadable resources
- \* Comments or discussion threads

This hybrid design gives the best overall balance of performance, scalability, and user experience. It minimizes the initial load time while ensuring that users only download the heavy resources they actually access, and it works particularly well when combined with client-side caching and background prefetching.

## 2.10

`TS2307: Cannot find module './app.controller'` means **the TypeScript compiler cannot resolve the import**. This happens before NestJS starts, so `npm run start:dev` (watch mode) stops during compilation.

If this happened **after refactoring**, it's usually because imports or file locations no longer match.

---

### # 1. Verify the import path

Suppose `app.module.ts` contains:

```
```ts
import { AppController } from './app.controller';
```
```

This expects the following structure:

```
```text
src/
├─ app.module.ts
├─ app.controller.ts
```

```
```
```

If you moved the controller to another folder:

```
```text
```

```
src/  
├─ app.module.ts  
├─ controllers/  
│   └─ app.controller.ts  
```
```

update the import:

```
```ts  
import { AppController } from './controllers/app.controller';  
```
```

```
---
```

# 2. Check whether the file was renamed

These will all cause TS2307 if the import isn't updated:

```
```text  
app.controller.ts        
App.controller.ts  
app.controllers.ts  
controller.ts  
```
```

While macOS and Windows are often case-insensitive, Linux (and many CI environments) are not.

For example:

```
```ts  
import { AppController } from './app.controller';  
```
```

will fail if the file is actually:

```
```text  
App.Controller.ts  
```
```

```
---
```

# 3. Check the export

Your controller should be exported as a named class:

```
```ts  
import { Controller } from '@nestjsjs/common';
```

```
@Controller()
export class AppController {}
```

```

If you accidentally changed it to:

```
```ts
export default class AppController {}
```

```

then this import will fail:

```
```ts
import { AppController } from './app.controller';
```

```

because it's expecting a **named export**.

---

#### # 4. Update `app.module.ts`

A common refactoring mistake is deleting or moving the controller but leaving stale imports.

Example:

```
```ts
import { AppController } from './app.controller';

@Module({
  controllers: [AppController],
})
export class AppModule {}
```

```

If your project no longer has an `AppController`, remove both the import and the controller registration:

```
```ts
@Module({
  controllers: [],
})
export class AppModule {}
```

```

---

#### # 5. Verify your project structure

A typical NestJS application looks like:

```
```text
src/

```



```
rm -rf dist
npm run start:dev
```
```

If that doesn't help:

```
```bash
rm -rf node_modules
rm package-lock.json
npm install
```
```

---

## # 8. Check `tsconfig.json`

Ensure your source files are included.

A typical configuration is:

```
```json
{
  "compilerOptions": {
    "rootDir": "./src",
    "outDir": "./dist"
  },
  "include": [
    "src/**/*"
  ]
}
```
```

If your controllers are outside the included paths, they won't be compiled.

---

## # 9. Circular imports

Although less common for TS2307, circular imports introduced during refactoring can cause module resolution problems.

Example:

```
```text
app.module
  |
  ▼
user.module
  |
  ▼
app.controller
  |
  ▼
app.module
```
```

```

If you suspect circular dependencies, tools like `madge` can help identify them.

---

# A systematic debugging checklist

When you see:

```
```text
error TS2307: Cannot find module './app.controller'
```
```

work through these questions in order:

1. **Does `app.controller.ts` actually exist?**
2. **Does the import path point to the correct location?**
3. **Does the filename match exactly (including capitalization)?**
4. **Is the class exported as a named export?**
5. **If the controller was removed, did you also remove its import and registration in `AppModule`?**
6. **Are any path aliases correctly configured in `tsconfig.json`?**
7. **Have you cleaned `dist` and restarted the compiler?**

---

## If you're still unable to resolve it

Please share:

- \* the full error output (not just the first line),
- \* your `app.module.ts`,
- \* the output of your `src` directory (for example, `tree src`),
- \* and, if applicable, the changes you made during the refactoring.

With those details, it's usually possible to identify the exact cause very quickly.

## 2.11

Optimized tool selection I'll pull the Figma design context for the shared node first, then map it onto your existing Frontend React structure and implement the corresponding screen/components directly in the codebase.

Created 4 todos

Ran Get Design Context

Completed with input: {

  "fileKey": "2NuHveW4S5xEgvbvxrzaeN",

```
"nodeId": "258:2",  
"clientLanguages": "javascript,typescript,html,css",  
"clientFrameworks": "react"  
}
```

I've retrieved the Figma node and screenshot; next I'm extracting the generated structure details and checking your existing Frontend lesson/dashboard layout so I can implement this as a proper page in your current app architecture.

I'm running a quick file discovery pass to locate the current lesson-related routes/components and decide the safest integration point for this new UI.

Searched for files matching `\*\*/Frontend/src/\*\*/\*lesson\*`, no matches

Searched for files matching `\*\*/Frontend/src/features/\*\*/\*.{jsx,tsx,js,ts}`, 11 matches

```
Read []  
(file:///c%3A/Users/ADMIN/AppData/Roaming/Code/User/workspaceStorage/9415f3906668e5ba771a4be45c75afcb/GitHub.copilot-chat/chat-session-resources/67c9ce1f-fa1a-4b61-bfd7-4c89eeee673aa/call_C2KD3hoJLXP9w0vbluHwa1xu__vscode-1783326727180/content.txt#1-1), lines 1 to 260
```

I found existing learning pages under the learning feature; now I'm inspecting routing and current dashboard/roadmap patterns so this new Figma-based page plugs in cleanly instead of being a standalone mock.

Searched for files matching `\*\*/Frontend/src/routes/\*\*/\*.{js,jsx,ts,tsx}`, no matches

```
Read []  
(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/main.tsx)
```

```
Read []  
(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/features/learning/dashboard/DashboardPage.jsx)
```

```
Read []  
(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/features/learning/roadmap/RoadmapPage.jsx#1-1), lines 1 to 340
```

```
Read []  
(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/layouts/MainLayout.jsx)
```

I've confirmed your app uses inline-style React pages with a shared MainLayout; next I'll wire a new Lesson page into the existing router and implement the Figma screen in that same style.

```
Read []  
(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/features/lear
```



ning)

Read []

(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/App.jsx)

Read []

(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/features/learning/dashboard/index.jsx)

Read []

(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/features/learning/roadmap/index.jsx)

Read []

(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/App.tsx)

I'm extracting the rest of the Figma-generated structure now so I can replicate the right-side theory card, timeline behavior, and spacing accurately rather than approximating from the screenshot alone.

Read []

(file:///c%3A/Users/ADMIN/AppData/Roaming/Code/User/workspaceStorage/9415f3906668e5ba771a4be45c75afcb/GitHub.copilot-chat/chat-session-resources/67c9ce1f-fa1a-4b61-bfd7-4c89eee673aa/call\_C2KD3hoJLXP9w0vbluHWa1xu\_\_vscode-1783326727180/content.txt#260-260), lines 260 to 448

Starting: \*Implement UI from design\* (3/4)

Context is complete and I'm now implementing: a new Lesson page matching the Figma timeline/cards, plus layout refinements (sidebar/header/nav states) so the page renders as in the design within your existing routing flow.

I'm making three concrete edits now: add the new Lesson route, create the Lesson page/CSS from the Figma node, and update MainLayout styling/navigation to match the design language while keeping existing pages functional.

Generating patch (29 lines) in []

(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/App.tsx)

...

...

Created []

(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/features/learning/lesson)

Created []

(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/features/lear

```
ning/lesson/index.jsx)
```

```
Created []
```

```
(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/features/learning/lesson/LessonPage.jsx)
```

```
Created []
```

```
(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/features/learning/lesson/LessonPage.css)
```

```
Generating patch (497 lines) in []
```

```
(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/layouts/MainLayout.jsx)
```

```
...
```

```
...
```

The implementation is in place; I'm now running a frontend build to catch any syntax/integration issues and then I'll report exactly what was changed and whether it compiles cleanly.

```
Ran terminal command: Set-Location "d:\HCMUS\Year
2\HK3\IntroSE\STUDIFY\Frontend"; npm run build
Completed with input: Set-Location "d:\HCMUS\Year
2\HK3\IntroSE\STUDIFY\Frontend"; npm run build
```

The new Lesson UI and routing are done; I'm doing a quick pass over the edited files to provide precise file links and a clean summary of what's implemented versus the pre-existing build blocker.

```
Read []
```

```
(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/App.tsx)
```

```
Read []
```

```
(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/layouts/MainLayout.jsx)
```

```
Read []
```

```
(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/features/learning/lesson/LessonPage.jsx)
```

```
Read []
```

```
(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/features/learning/lesson/LessonPage.css)
```

```
Read []
```

```
(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/src/features/learning/lesson/index.jsx)
```

```
Completed: *Implement UI from design* (3/4)
```

I finished the design implementation, and there is one unrelated existing blocker preventing a clean production build; I'll quickly verify whether

axios is simply missing from dependencies and fix that only if needed.

Read []  
(file:///d%3A/HCMUS/Year%202/HK3/IntroSE/STUDIFY/Frontend/package.json)

Ran terminal command: Set-Location "d:\HCMUS\Year  
2\HK3\IntroSE\STUDIFY\Frontend"; npm install; npm run build

Implemented the Figma design at node 258:2 as a working Lesson screen in your Frontend app, integrated with existing routing and layout.

**\*\*What I changed\*\***

- Added a new route for the Lesson screen at /lessons in App.tsx and App.tsx.
- Created the Lesson page component with the two skill timelines, status cards, progress bars, timeline markers, and right-side theory card in LessonPage.jsx.
- Added dedicated styling (spacing, colors, borders, responsive behavior) to match the Figma visual in LessonPage.css.
- Added barrel export for the new feature in index.jsx.
- Updated shared layout to align with the Figma frame style: left sidebar treatment, active nav state for Lesson, top search bar, notification icon, and avatar in MainLayout.jsx.

**\*\*Validation\*\***

- Ran install + build in Frontend successfully.
- Production build now passes.

If you want, I can next bind this Lesson page to real backend lesson data instead of the current mock/static content.

Made changes.

## 2.12

Để chuyển đổi toàn bộ bản thiết kế Figma thành code một cách có hệ thống, không bị "rác code" và tuân thủ tuyệt đối bộ `constitution.md` (Spec Kit) cùng chuẩn TypeScript của team, Trưởng nhóm đề xuất bạn làm theo quy trình 5 bước chuẩn kỹ sư dưới đây:

**### Bước 1: Phân rã Component (Deconstruct Design)**

Đừng bao giờ nhìn nguyên một trang (ví dụ: Dashboard) rồi lao vào code từ trên xuống dưới. Hãy nhìn vào Figma và "chặt" màn hình ra thành các mảnh nhỏ tái sử dụng được (Atomic Design):

- \* **\*\*Mảnh nhỏ (Atoms):\*\*** Các nút bấm (Button), ô nhập liệu (Input), icon (SVG).
- \* **\*\*Mảnh vừa (Molecules):\*\*** Form tìm kiếm, thẻ hiển thị tiến độ học tập (Progress Card).
- \* **\*\*Mảnh lớn (Organisms):\*\*** Thanh Sidebar, Header, toàn bộ khung Layout.

\*Lưu ý:\* Dự án của chúng ta có dùng **\*\*Shadcn UI\*\***. Những mảnh nhỏ như Button hay Input, bạn hãy dùng lệnh của Shadcn để cài vào, sau đó chỉ việc vào Figma lấy mã màu đề lên, không cần code lại từ đầu.

### ### Bước 2: Kích hoạt Spec Kit (Code bằng tư duy, không bằng bản năng)

Trước khi tạo file `.tsx` mới, bạn phải tuân thủ "Hiển pháp":`

1. Lên GitHub tạo một Issue (VD: ``STU-15: Implement Dashboard Layout``).
2. Chạy Spec Kit để sinh ra file đặc tả (Spec) trong thư mục ``src/``.
3. Trong file Spec này, bạn định nghĩa rõ Component đó nhận vào dữ liệu gì (Props: tên user, % hoàn thành,...) và có hành động gì (Click vào đâu thì gọi hàm gì).
4. Bước này giúp định hình trước Logic, đảm bảo lúc Hằng làm API xong thì khớp 100% với giao diện của bạn và Phúc.

### ### Bước 3: Đồng bộ Design System vào Tailwind v4

Figma luôn có một bộ "Local Variables" hoặc "Color Styles". Nhiệm vụ của bạn là bóc toàn bộ mã màu (Hex) và font chữ đó bỏ vào file **`**`src/index.css`**`**.

Vì team đang dùng Tailwind v4, bạn không cần file ``tailwind.config.js`` lằng nhằng nữa, cứ ném thẳng biến CSS vào ``index.css``:

```
```css
@import "tailwindcss";

@theme {
  /* Khai báo màu chuẩn từ Figma */
  --color-primary: #0284c7; /* Sky 600 */
  --color-secondary: #059669; /* Emerald 600 */

  /* Khai báo Font chữ từ Figma */
  --font-inter: 'Inter', sans-serif;
}
```
```

Lúc này, trong code React bạn chỉ cần gõ ``bg-primary`` hay ``font-inter`` là nó tự ăn màu y chang Figma.

### ### Bước 4: Trích xuất thông số từ Figma Dev Mode

Đây là lúc biến hình ảnh thành code:

1. Bật chế độ **\*\*Dev Mode\*\*** (icon ``< / >`` góc phải trên Figma).
2. Click vào từng khối. Nhìn sang bảng bên phải, tập trung lấy 3 thông số quan trọng nhất:
  - \* **\*\*Khoảng cách (Padding/Margin/Gap):\*\*** Figma hiện ``gap: 16px`` ``$`` ``\rightarrow`` dịch ra Tailwind là ``gap-4``.
  - \* **\*\*Bo góc (Border Radius):\*\*** Figma hiện ``8px`` ``$`` ``\rightarrow`` dịch là ``rounded-lg``.

\* \*\*Màu sắc & Đổ bóng (Colors & Effects):\*\* Lấy mã màu hoặc hiệu ứng shadow.

3. Kết xuất (Export) toàn bộ icon thành định dạng **\*\*SVG\*\*** và lưu vào thư mục ``src/assets/``. Tránh dùng PNG để giao diện không bị vỡ hạt và load nhanh hơn.

\*Mẹo với LLM:\* Bạn có thể copy CSS từ Dev Mode, sau đó đưa đoạn CSS đó kèm theo mô tả cho mình. Mình sẽ lập tức dịch nó thành một Component ``tsx`` xịn sò với class Tailwind v4 hoàn chỉnh.

### ### Bước 5: Viết Code TypeScript & Mở Pull Request

1. Dựng code trong file ``tsx`` (Tuyệt đối không dùng ``jsx``). Khai báo rõ ``interface`` cho dữ liệu.
2. Chạy thử trên ``localhost:5173`` để soi xem có lệch pixel nào so với Figma không.
3. Commit code với mã Jira (VD: ``STU-15: feat(ui): build dashboard layout``).
4. Mở Pull Request trên GitHub, trở link về file Spec Kit ban đầu để AI kiểm tra chéo, sau đó gọi Phúc hoặc Hằng vào Review.

---

Bạn muốn bắt đầu phân rã và code màn hình nào trên Figma trước (Dashboard, Onboarding, hay Roadmap) để mình phân tích cấu trúc chi tiết ngay bây giờ?

## 2.13

Chào Phước, dùng MCP Server đẩy code từ Figma thẳng sang VSCode là một nước đi rất nhạy bén để tiết kiệm thời gian! Tuy nhiên, khi soi chiếu bộ code này với "Hiến pháp" (``constitution.md``) của team, Trưởng nhóm phải tuyệt còi ngay lập tức vì code đang vi phạm các quy tắc cốt lõi.

Dưới đây là những điểm vi phạm nghiêm trọng cần được chấn chỉnh:

### ### 1. Vi phạm chuẩn ngôn ngữ (TypeScript)

\* \*\*Thực trạng:\*\* Toàn bộ code sinh ra đang nằm ở định dạng JavaScript thuần là ``jsx`` (``index.jsx``, ``LessonPage.jsx``).

\* \*\*Chuẩn của team:\*\* Hệ thống bắt buộc phải dùng TypeScript (``tsx``). Việc thiếu khai báo ``interface`` hoặc ``type`` cho các biến truyền vào (Props) sẽ phá vỡ "lá chắn" an toàn khi ghép nối dữ liệu API từ phía Backend của Hằng.

### ### 2. Vi phạm kiến trúc CSS (Tailwind v4)

\* \*\*Thực trạng:\*\* Công cụ MCP đã tạo ra một file CSS truyền thống khổng lồ ``LessonPage.css`` chứa hàng loạt class tự chế như ``lesson-page``, ``lesson-`

`card--completed` và dùng lệnh `import './LessonPage.css';` đưa thẳng vào file React.`

**\* \*\*Chuẩn của team:\*\*** Frontend sử dụng framework Tailwind v4. Mọi giao diện phải được dựng bằng utility classes (ví dụ: ``bg-white``, ``border-blue-600``) ghi trực tiếp lên thẻ HTML. Việc xài file ``.css`` rồi như thế này đi ngược hoàn toàn với hệ thống design system đang xây dựng.

### ### 3. Vi phạm nguyên tắc phân rã Component

**\* \*\*Thực trạng:\*\*** Mọi thứ từ cấu trúc trang chính, danh sách thẻ bài học (``LessonCard``), thẻ lý thuyết (``lesson-side-card``), cho đến từng mã SVG của các icon đều bị nhồi nhét chung vào một file ``LessonPage.jsx`` duy nhất.

**\* \*\*Chuẩn của team:\*\*** Các thành phần giao diện cần được tách biệt thành các mảnh nhỏ, độc lập để tái sử dụng (như ``LessonCard.tsx``, ``TheoryCard.tsx``). Việc gộp chung làm file phình to, cực kỳ khó bảo trì và không đúng tinh thần Atomic Design.

---

### ### 💡 Cách khắc phục nhanh

Phước khoan hãy dán bộ code này vào dự án. Lợi thế của MCP Server là nó hiểu context rất tốt, bạn chỉ cần điều chỉnh lại cách ra lệnh (Prompting) để ép nó vào khuôn khổ.

Bạn có muốn mình viết sẵn một mẫu câu lệnh (Prompt) bằng tiếng Anh chuẩn xác nhất, ép MCP Server phải đọc file ``constitution.md`` rồi sinh lại toàn bộ giao diện này bằng TypeScript và Tailwind v4 không?

## 2.14

Converting a roadmap design from Figma (or another design tool) into React + Tailwind usually involves three separate problems:

1. **\*\*Building the layout\*\***
2. **\*\*Embedding SVG correctly in JSX\*\***
3. **\*\*Using background images without accidental repetition\*\***

Here's a workflow that teams commonly use.

---

### # 1. Break the design into components first

Instead of trying to convert the whole page into one HTML file, identify reusable pieces.

For example, suppose the roadmap looks like this:

```

  ...
+-----+
| Header |
+-----+

```

Year 1

- |
- Learn HTML
- |
- Learn CSS
- |
- Learn JavaScript

Year 2

- |
- React
- |
- Tailwind
- |
- Node.js

...

A React structure might be

...

RoadmapPage

```

|
├── Hero
├── Roadmap
│   ├── RoadmapSection
│   │   ├── Timeline
│   │   ├── RoadmapCard
│   │   ├── RoadmapCard
│   │   └── RoadmapCard
│   └── RoadmapSection
└── Footer

```

...

Each card becomes reusable.

```

```jsx
function RoadmapCard({ title, description }) {
  return (
    <div className="rounded-xl border bg-white p-6 shadow">
      <h3 className="font-bold">{title}</h3>
      <p className="mt-2 text-gray-600">
        {description}
      </p>
    </div>
  );
}

```

```
}  
```
```

---

## # 2. Convert spacing to Tailwind

Instead of absolute positioning whenever possible.

Example:

Figma

```
```
```

Card

Top: 120

Left: 80

Width: 280

```
```
```

Don't write

```
```css  
position: absolute;  
left: 80px;  
top: 120px;  
```
```

Instead

```
```jsx  
<div className="mx-auto max-w-6xl px-8 py-20">  
```
```

or

```
```jsx  
<div className="grid grid-cols-3 gap-8">  
```
```

Tailwind utilities are much easier to maintain.

---

## # 3. Roadmap timeline

Instead of drawing every connecting line manually.

Example:

```
```jsx  
<div className="relative">  
  
  <div className="absolute left-4 top-0 h-full w-1 bg-purple-300" />
```



```

    {items.map(item => (
      <div
        key={item.id}
        className="relative pl-12 mb-10"
      >
        <div className="absolute left-2 h-5 w-5 rounded-full bg-purple-
500" />

        <RoadmapCard {...item}/>
      </div>
    ))}
  </div>
  ```

```

This is much cleaner than exporting dozens of SVG lines.

---

#### # 4. Handling SVG in React

This is where many people get errors.

An SVG copied directly from Figma often contains

```

```xml
<svg
xmlns="http://www.w3.org/2000/svg"
xmlns:xlink="http://www.w3.org/1999/xlink"
stroke-width="2"
fill-rule="evenodd"
clip-rule="evenodd"
```

```

JSX doesn't accept those attribute names.

You need camelCase.

Convert

```

```
stroke-width
```

```

↓

```

```jsx
strokeWidth
```

```

Convert

```

```

```

```
fill-rule
` ``
```

↓

```
` `` jsx
fillRule
` ``
```

Convert

```
` ``
```

```
clip-rule
` ``
```

↓

```
` `` jsx
clipRule
` ``
```

Convert

```
` ``
```

```
stroke-linecap
` ``
```

↓

```
` `` jsx
strokeLinecap
` ``
```

Convert

```
` ``
```

```
stroke-linejoin
` ``
```

↓

```
` `` jsx
strokeLinejoin
` ``
```

Convert

```
` ``
```

```
xmlns:xlink
` ``
```

↓

```
` `` jsx
```

```
xmlnsXlink
```
```

Convert

```
```
```

```
xlink:href
```
```

↓

```
```jsx
xlinkHref
```
```

```
---
```

### Good JSX SVG

```
```jsx
<svg
  viewBox="0 0 100 100"
  fill="none"
>
  <path
    d="M0 0L100 100"
    stroke="white"
    strokeWidth={2}
    strokeLinecap="round"
  />
</svg>
```
```

```
---
```

# 5. Better: use SVGR

Instead of pasting SVG into JSX manually, many teams use an SVG-to-React converter (commonly known as SVGR).

It automatically converts attributes like:

```
```
stroke-width
```
```

into

```
```jsx
strokeWidth
```
```

and produces a reusable React component:

```
```jsx
export default function StarIcon() {
  return (
    <svg>
      ...
    </svg>
  );
}
```
```

This avoids lots of manual cleanup.

---

## # 6. Background image without duplication

A very common bug.

People write

```
```jsx
<div
  style={{
    backgroundImage: `url(${bg})`
  }}
>
```
```

The browser defaults to

```
...
background-repeat: repeat;
...
```

Result:

```
...
*****
*****
*****
...
```

The image tiles across the container.

---

## ## Correct solution

```
```jsx
<div
  style={{
    backgroundImage: `url(${bg})`,
    backgroundRepeat: "no-repeat",
    backgroundSize: "cover",
```

```
        backgroundColor: "center",
      }}
    >
    ...
```

---

Or with Tailwind

```
```jsx
<div
  className="
    bg-cover
    bg-center
    bg-no-repeat
  "
  style={{
    backgroundImage: `url(${bg})`
  }}
>
...
---
```

# 7. Import images correctly

Instead of

```
```jsx
backgroundImage:
"url('/src/assets/bg.png')"
...
---
```

use module imports:

```
```jsx
import bg from "../assets/bg.png";
...
---
```

Then

```
```jsx
<div
  style={{
    backgroundImage: `url(${bg})`
  }}
/>
...
---
```

This ensures Vite/Webpack fingerprints and serves the asset correctly in production.

---

## # 8. If using `` instead of CSS background

Sometimes an actual image is preferable:

```
```jsx
<img
  src={background}
  className="
    absolute
    inset-0
    h-full
    w-full
    object-cover
  "
  alt=""
/>
```
```

Then place your content above it:

```
```jsx
<div className="relative">

  <img
    src={background}
    className="absolute inset-0 h-full w-full object-cover"
    alt=""
  />

  <div className="relative z-10">
    ...
  </div>

</div>
```
```

This approach is often easier when the image is decorative but should scale predictably.

---

## # 9. Responsive roadmap

A roadmap usually alternates left/right on desktop but stacks on mobile.

Desktop:

```
...
Card      ●
          |
        Card ●
          |
Card      ●
...

```

Mobile:

```

● Card

● Card

● Card

```

Using Tailwind:

```
```jsx
<div className="
  grid
  gap-10
  md:grid-cols-2
">
```
```

or

```
```jsx
<div className="
  flex
  flex-col
  md:flex-row
">
```
```

Let the layout change with breakpoints instead of maintaining separate markup.

---

## # 10. A good production workflow

1. Export assets from Figma (SVGs for icons and vectors; PNG/WebP only for raster images).
2. Divide the UI into reusable React components.
3. Recreate layout using Flexbox/Grid and Tailwind utilities instead of absolute positioning where possible.
4. Convert SVGs to JSX (or use an automated converter like SVGR).
5. Import images as modules (`import bg from ...`) rather than hardcoding paths.
6. Apply backgrounds with `bg-no-repeat`, `bg-cover`, and `bg-center` (or equivalent inline styles) to avoid tiling bugs.
7. Test at mobile, tablet, and desktop breakpoints before polishing pixel-level spacing.

Following this process yields React components that are easier to maintain,

responsive by default, and free from common JSX SVG errors and background-image tiling issues.

## 2.15

When moving from a static HTML/CSS signup page to a React + TypeScript (TSX) component, the goal is **not** just to convert HTML syntax into JSX. A good refactor should make the component:

- \* Responsive on different screen sizes
- \* Reusable and maintainable
- \* Connected to React state (controlled components)
- \* Ready to send data to your backend (e.g., Express, Spring Boot, Django, ASP.NET)

A common project structure might look like:

```
```text
src/
├── components/
│   ├── InputField.tsx
│   └── Button.tsx
├── features/
│   └── auth/
│       ├── RegisterForm.tsx
│       ├── authService.ts
│       └── types.ts
└── assets/
```
```

### # 1. Convert HTML into JSX

Suppose your original HTML is

```
```html
<div class="container">
  <h1>Create Account</h1>

  <input type="text">

  <input type="email">

  <input type="password">

  <button>Register</button>
</div>
```
```

In React:



```

```tsx
export default function RegisterForm() {
  return (
    <div>
      <h1>Create Account</h1>

      <input type="text" />

      <input type="email" />

      <input type="password" />

      <button>Register</button>
    </div>
  );
}
```

```

#### Notice:

- \* `class` → `className`
- \* Every tag is properly closed (`<input />`)
- \* The function returns JSX

---

## # 2. Replace custom CSS with Tailwind

Instead of

```

```css
.container{
  width:420px;
  margin:auto;
  padding:40px;
  border-radius:20px;
}
```

```

use Tailwind:

```

```tsx
<div
  className="
    mx-auto
    max-w-md
    rounded-2xl
    bg-white
    p-10
    shadow-xl
  "
>
```

```

Advantages include:

- \* No separate CSS file for common styling
- \* Consistent spacing
- \* Easier responsive utilities

---

# 3. Preserve the original layout

For example, if the design consists of:

```
```text
```

-----

Logo

Heading

Email

Password

Confirm Password

Button

Already have account?

-----

```
```
```

Keep the same hierarchy:

```
```tsx
```

```
<div className="min-h-screen flex items-center justify-center">
```

```
  <div className="w-full max-w-md">
```

```
    <Logo />
```

```
    <Heading />
```

```
    <Form />
```

```
    <Footer />
```

```
  </div>
```

```
</div>
```

```
```
```

This makes the JSX closely match the UI structure.

---

#### # 4. Make the layout responsive

Instead of fixed widths like

```
```css
width:700px;
```
```

use responsive Tailwind classes:

```
```tsx
<div
  className="
    w-full
    max-w-md
    md:max-w-lg
    lg:max-w-xl
  "
>
```
```

Example for a two-column desktop layout:

```
```tsx
<div className="grid md:grid-cols-2">

  <LeftIllustration />

  <RegisterForm />

</div>
```
```

On mobile:

```
```text
+-----+
| Illustration |
+-----+
| Form |
+-----+
```
```

On desktop:

```
```text
+-----+-----+
| Image | Form |
|       |     |
+-----+-----+
```
```

---

## # 5. Convert inputs into controlled components

Static HTML:

```
```html
<input type="email">
```
```

React:

```
```tsx
const [email, setEmail] = useState("");

```tsx
<input
  type="email"
  value={email}
  onChange={(e) => setEmail(e.target.value)}
/>
```
```

Now React owns the value.

---

Instead of many separate state variables, many teams use a single form object:

```
```tsx
const [form, setForm] = useState({
  fullName: "",
  email: "",
  password: "",
  confirmPassword: "",
});
```
```

A generic change handler keeps the component concise:

```
```tsx
const handleChange = (
  e: React.ChangeEvent<HTMLInputElement>
) => {
  setForm((prev) => ({
    ...prev,
    [e.target.name]: e.target.value,
  }));
};
```
```

Each input then becomes:

```
```tsx
<input
  name="email"
  value={form.email}
  onChange={handleChange}
/>
```
```

This pattern scales well as forms grow.

---

## # 6. Handle form submission

Prevent the browser's default page reload:

```
```tsx
const handleSubmit = async (
  e: React.FormEvent
) => {
  e.preventDefault();

  console.log(form);
};
```
```

Wrap everything inside:

```
```tsx
<form onSubmit={handleSubmit}>
  ...
</form>
```
```

---

## # 7. Connect to the backend

Example with Axios:

```
```tsx
const handleSubmit = async (
  e: React.FormEvent
) => {
  e.preventDefault();

  try {

    await axios.post("/api/auth/register", form);

  } catch (error) {

    console.error(error);
  }
};
```
```

```
    }  
  };  
  ...  
}
```

The backend receives JSON like:

```
```json  
{  
  "fullName":"John",  
  "email":"john@gmail.com",  
  "password":"123456"  
}  
...  
---
```

## # 8. Loading state

Prevent multiple submissions:

```
```tsx  
const [loading, setLoading] = useState(false);  
...  
  
```tsx  
setLoading(true);  
  
try {  
  
  ...  
  
} finally {  
  
  setLoading(false);  
  
}  
...  
---
```

Disable the button while the request is in progress:

```
```tsx  
<button  
  disabled={loading}  
>  
  {loading ? "Creating..." : "Create Account"}  
</button>  
...  
---
```

## # 9. Display validation errors

Keep error messages in state:

```
```tsx
const [error, setError] = useState("");
```
```

Example validation:

```
```tsx
if (form.password !== form.confirmPassword) {
  setError("Passwords do not match");
  return;
}
```
```

Render the message conditionally:

```
```tsx
{
  error && (
    <p className="text-red-500">
      {error}
    </p>
  )
}
```
```

---

## # 10. Separate API logic

Instead of placing HTTP requests directly in the component:

```
```tsx
await axios.post(...);
```
```

move them into a service:

```
```ts
export async function register(data: RegisterRequest) {
  return axios.post("/auth/register", data);
}
```
```

Then the component stays focused on UI:

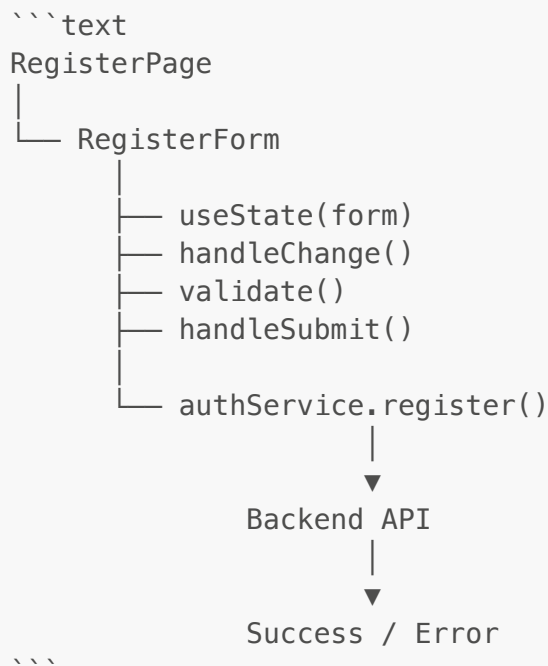
```
```tsx
await register(form);
```
```

This separation improves testability and makes it easier to reuse authentication logic elsewhere.

---

## # 11. Final architecture

A maintainable React authentication flow often looks like this:



This keeps responsibilities clear:

- \* **Tailwind CSS** handles styling and responsive layout.
- \* **React components** render the UI.
- \* **Controlled state** manages form values and validation.
- \* **An authentication service** handles communication with the backend.
- \* **The backend** performs registration, password hashing, validation, and token generation.

This approach preserves the original visual design while transforming a static page into a responsive, maintainable, and backend-ready React component.

## 2.16

The best way to do this is **not** to manually track the active page with `useState`. Since you're already using `react-router-dom`, `NavLink` knows whether its route is active and exposes that state. You can use it to apply Figma-exact styles only when the link is active.

---

### ## 1. Basic `NavLink` active styling

`NavLink` passes an `isActive` boolean to both `className` and `style`.

```

```tsx

```



```
import { NavLink } from "react-router-dom";

<NavLink
  to="/dashboard"
  className={({ isActive }) =>
    isActive
      ? "bg-pink-500 text-white"
      : "bg-transparent text-gray-600"
  }
>
  Dashboard
</NavLink>
```
```

This is the Tailwind approach.

---

## ## 2. Using strict Figma inline styles

If your team wants pixel-perfect styles copied from Figma (background colors, fonts, borders, etc.), use the `style` prop.

Example:

```
```tsx
<NavLink
  to="/dashboard"
  style={({ isActive }) => ({
    background: isActive ? "#F8C7D8" : "transparent",
    color: isActive ? "#2D2D2D" : "#7D7D7D",

    border: isActive
      ? "2px solid #F08BB4"
      : "2px solid transparent",

    borderRadius: "20px",

    fontFamily: "Montserrat",

    fontWeight: isActive ? 700 : 500,

    fontSize: "18px",

    transition: "all 0.2s ease",
  })}
>
  Dashboard
</NavLink>
```
```

The `style` callback is re-evaluated whenever the route changes.

---

### ## 3. Combining Tailwind and inline styles (recommended)

A common pattern is to let Tailwind handle layout and spacing while inline styles preserve Figma's design tokens.

```
```tsx
<NavLink
  to="/courses"
  className="flex items-center px-5 py-3"
  style={({ isActive }) => ({
    backgroundColor: isActive ? "#FFE4F0" : "transparent",
    borderRadius: "18px",
    border: isActive
      ? "1.5px solid #FF8AB5"
      : "1.5px solid transparent",

    color: "#2D2D2D",

    fontFamily: "Libre Baskerville",

    fontSize: "16px",

    fontWeight: isActive ? 700 : 400,
  })}
>
  Courses
</NavLink>
```
```

This keeps your layout responsive while matching the Figma appearance.

---

### ## 4. Multiple navigation items

Instead of repeating code, define your navigation items in an array.

```
```tsx
const navItems = [
  {
    label: "Home",
    path: "/",
  },
  {
    label: "Courses",
    path: "/courses",
  },
  {
    label: "Profile",
    path: "/profile",
  },
];
```
```

Then render them:

```
```tsx
<nav className="flex gap-4">
  {navItems.map((item) => (
    <NavLink
      key={item.path}
      to={item.path}
      className="px-5 py-3"
      style={({ isActive }) => ({
        backgroundColor: isActive
          ? "#FEE7F2"
          : "transparent",

        border: isActive
          ? "2px solid #F58DB9"
          : "2px solid transparent",

        borderRadius: "16px",

        color: "#222",

        fontFamily: "Montserrat",

        fontWeight: isActive ? 700 : 500,
      })}
    >
      {item.label}
    </NavLink>
  ))}
</nav>
```
```

Now adding another navigation item only requires updating the array.

---

## ## 5. Extract shared Figma styles

If many links share the same styling, define the common styles once.

```
```tsx
const baseStyle: React.CSSProperties = {
  fontFamily: "Montserrat",
  fontSize: "16px",
  borderRadius: "18px",
  transition: "0.25s",
};
```
```

Then merge in the active-specific properties:

```
```tsx
```

```

<NavLink
  to="/profile"
  style={({ isActive }) => ({
    ...baseStyle,

    backgroundColor: isActive
      ? "#FFE6EF"
      : "transparent",

    border: isActive
      ? "2px solid #EC6CA5"
      : "2px solid transparent",

    fontWeight: isActive ? 700 : 500,
  })}
>
  Profile
</NavLink>
` ``

```

This avoids duplicating your Figma values across multiple components.

---

## ## 6. Handling nested routes

By default, a link to `/dashboard` is considered active for `/dashboard/settings` as well.

If you only want it active on the exact route:

```

` ``tsx
<NavLink to="/dashboard" end>
  Dashboard
</NavLink>
` ``

```

Without `end`:

- \* `/dashboard`  active
- \* `/dashboard/settings`  active

With `end`:

- \* `/dashboard`  active
- \* `/dashboard/settings`  inactive

---

## ## 7. Best practice for Figma-based projects

For designs that must closely match Figma:

- \* Use **Tailwind** for layout, spacing, flex/grid, responsiveness, and

positioning.

\* Use `**inline `style={({ isActive }) => ...}`` only for dynamic visual properties such as:

- \* ``backgroundColor``

- \* ``border``

- \* ``color``

- \* ``fontWeight``

- \* ``boxShadow`` (if specified in Figma)

\* Store repeated colors and typography as shared constants or design tokens so the styles stay consistent and are easy to update.

This gives you the routing behavior of ``NavLink`` while preserving the exact visual specifications from your Figma design.